Proyecto Analítica de campañas de marketing en Banco Santander

Integrantes: Adrian Bajaña , Miguel Molina, María José Ramírez

In [106…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración visual
plt.rcParams["figure.figsize"] = (8, 5)
sns.set_style("whitegrid")
```

In [107…

```
df = pd.read_csv("informacion_campana_bancaria_2026.csv")
df.head()
```

Out[107…

	age	job	marital	education	default	housing	loan	contact	m
0	30	blue-collar	single	basic.9y	no	yes	no	telephone	
1	58	retired	married	professional.course	no	yes	no	cellular	
2	48	technician	married	professional.course	no	yes	yes	cellular	
3	39	blue-collar	single	basic.9y	no	yes	no	cellular	
4	56	services	married	basic.4y	NaN	no	no	telephone	

5 rows × 21 columns

Revisión inicial del dataset

Realizamos una inspección general del dataset para conocer su tamaño, estructura, tipos de datos y primeras observaciones antes de iniciar el proceso de limpieza y análisis.

In [108…

```
print("Dimensiones del dataset:", df.shape)
print("\nColumnas:")
print(df.columns.tolist())

print("\nTipos de datos:")
print(df.dtypes)

print("\nInformación general:")
df.info()
```

```
Dimensiones del dataset: (41388, 21)

Columnas:
['age', 'job', 'marital', 'education', 'default', 'housing', 'loan', 'cont
act', 'month', 'day_of_week', 'duration', 'campaign', 'pdays', 'previous',
'poutcome', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3
m', 'nr.employed', 'y']

Tipos de datos:
age                 int64
job                   str
marital               str
education             str
default               str
housing               str
loan                  str
contact               str
month                 str
day_of_week           str
duration            int64
campaign            int64
pdays               int64
previous            int64
poutcome              str
emp.var.rate      float64
cons.price.idx    float64
cons.conf.idx     float64
euribor3m         float64
nr.employed       float64
y                     str
dtype: object

Información general:
<class 'pandas.DataFrame'>
RangeIndex: 41388 entries, 0 to 41387
Data columns (total 21 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             41388 non-null  int64
 1   job             41054 non-null  str
 2   marital         41307 non-null  str
 3   education       39644 non-null  str
 4   default         32743 non-null  str
 5   housing         40392 non-null  str
 6   loan            40392 non-null  str
 7   contact         41388 non-null  str
 8   month           41388 non-null  str
 9   day_of_week     41388 non-null  str
 10  duration        41388 non-null  int64
 11  campaign        41388 non-null  int64
 12  pdays           41388 non-null  int64
 13  previous        41388 non-null  int64
 14  poutcome        41388 non-null  str
 15  emp.var.rate    41388 non-null  float64
 16  cons.price.idx  41388 non-null  float64
 17  cons.conf.idx   41388 non-null  float64
 18  euribor3m       41388 non-null  float64
 19  nr.employed     41388 non-null  float64
 20  y               41388 non-null  str
```

```
dtypes: float64(5), int64(5), str(11)
memory usage: 6.6 MB
```

In [109…

```
print("\nPrimeras 5 filas:")
display(df.head())

print("\nÚltimas 5 filas:")
display(df.tail())
```

```
Primeras 5 filas:
```

	age	job	marital	education	default	housing	loan	contact	mo
0	30	blue-collar	single	basic.9y	no	yes	no	telephone	
1	58	retired	married	professional.course	no	yes	no	cellular	
2	48	technician	married	professional.course	no	yes	yes	cellular	
3	39	blue-collar	single	basic.9y	no	yes	no	cellular	
4	56	services	married	basic.4y	NaN	no	no	telephone	

5 rows × 21 columns

```
Últimas 5 filas:
```

	age	job	marital	education	default	housing	loan	contact	mont
41383	21	unemployed	married	high.school	no	yes	yes	cellular	ju
41384	24	blue-collar	single	basic.9y	no	yes	no	cellular	ju
41385	21	student	single	basic.4y	no	no	no	cellular	ma
41386	52	admin.	married	high.school	no	no	no	cellular	no
41387	40	services	married	basic.6y	no	no	no	cellular	no

5 rows × 21 columns

Análisis de duplicados

Revisamos si existen registros duplicados en el dataset, ya que estos podrían sesgar los resultados del análisis descriptivo y afectar las conclusiones del proyecto.

In [110…

```
duplicados = df.duplicated().sum()
print("Número de filas duplicadas:", duplicados)
```

```
Número de filas duplicadas: 12
```

In [111…

```
df = df.drop_duplicates()
print("Nueva dimensión luego de eliminar duplicados:", df.shape)
```

```
Nueva dimensión luego de eliminar duplicados: (41376, 21)
```

Valores nulos

Revisamos la existencia de valores nulos explícitos en el dataset. Además consideramos que en varias variables categóricas los valores faltantes se encuentran codificados como `"unknown"` por lo que vamos a analizar dichos valores.

In [112…

```
print("Valores nulos por columna:")
print(df.isnull().sum())

print("\nPorcentaje de nulos por columna:")
print((df.isnull().sum() / len(df) * 100).round(2))
```

```
Valores nulos por columna:
age                  0
job                334
marital             81
education         1743
default           8644
housing            996
loan               996
contact              0
month                0
day_of_week          0
duration             0
campaign             0
pdays                0
previous             0
poutcome             0
emp.var.rate         0
cons.price.idx       0
cons.conf.idx        0
euribor3m            0
nr.employed          0
y                    0
dtype: int64

Porcentaje de nulos por columna:
age                0.00
job                0.81
marital            0.20
education          4.21
default           20.89
housing            2.41
loan               2.41
contact            0.00
month              0.00
day_of_week        0.00
duration           0.00
campaign           0.00
pdays              0.00
previous           0.00
poutcome           0.00
emp.var.rate       0.00
cons.price.idx     0.00
cons.conf.idx      0.00
euribor3m          0.00
nr.employed        0.00
y                  0.00
dtype: float64
```

In [113…
```
columnas_categoricas = df.select_dtypes(include=["object","string"]).colu

for col in columnas_categoricas:
    cantidad_unknown = (df[col] == "unknown").sum()

    if cantidad_unknown > 0:
        print(f"{col}: {cantidad_unknown} registros 'unknown' ({cantidad_
```

Validación de variables discretas

Revisaremos si las variables que conceptualmente deben ser discretas están almacenadas como decimales. En caso convertiremos a enteros para mantener consistencia en el análisis.

In [114…
```
print(df[["age", "duration", "campaign", "pdays", "previous"]].dtypes)
```

```
age         int64
duration    int64
campaign    int64
pdays       int64
previous    int64
dtype: object
```

Revisión de observaciones válidas

Revisaremos si los valores de ciertas variables numéricas se encuentran dentro de rangos razonables, con el fin de identificar posibles errores de captura o datos atípicos que deban considerarse en el análisis.

In [115…
```
print("Edad mínima y máxima:", df["age"].min(), df["age"].max())
print("Duración mínima y máxima:", df["duration"].min(), df["duration"].m
print("Campaign mínima y máxima:", df["campaign"].min(), df["campaign"].m
print("Previous mínima y máxima:", df["previous"].min(), df["previous"].m
print("Pdays mínima y máxima:", df["pdays"].min(), df["pdays"].max())
```

```
Edad mínima y máxima: 17 98
Duración mínima y máxima: 0 4918
Campaign mínima y máxima: 1 56
Previous mínima y máxima: 0 7
Pdays mínima y máxima: 0 999
```

In [116…
```
df[["age", "duration", "campaign", "pdays", "previous"]].describe()
```

Out[116…

	age	duration	campaign	pdays	previous
count	41376.000000	41376.000000	41376.000000	41376.000000	41376.000000
mean	40.025836	257.067189	2.569992	962.497051	0.172854
std	10.421611	259.297541	2.775531	186.858704	0.494654
min	17.000000	0.000000	1.000000	0.000000	0.000000
25%	32.000000	101.000000	1.000000	999.000000	0.000000
50%	38.000000	179.000000	2.000000	999.000000	0.000000
75%	47.000000	318.000000	3.000000	999.000000	0.000000
max	98.000000	4918.000000	56.000000	999.000000	7.000000

Validación de llamadas no contestadas

Se identificaron 204 registros con duración igual a cero, lo que significa que la llamada no fue atendida por el cliente.

Estos casos representan una proporción muy pequeña del dataset, por lo que lo mantendremos únicamente para el análisis descriptivo.

In [117…

```
# Verificar llamadas no contestadas
llamadas_no_contestadas = (df["duration"] == 0).sum()

print("Número de llamadas no contestadas:", llamadas_no_contestadas)
```

```
Número de llamadas no contestadas: 204
```

In [118…

```
# Verificar inconsistencias: duración 0 pero cliente acepta

inconsistentes = df[(df["duration"] == 0) & (df["y"] == "yes")]

print("Número de registros inconsistentes:", inconsistentes.shape[0])

inconsistentes.head()
```

```
Número de registros inconsistentes: 25
```

Out[118…

	age	job	marital	education	default	housing	loan	contact	mo
1051	28	admin.	married	university.degree	no	yes	no	cellular	
1670	39	admin.	married	basic.6y	NaN	no	no	cellular	
3923	73	retired	married	basic.4y	no	no	no	cellular	
4031	28	student	single	high.school	no	yes	no	cellular	
4450	39	admin.	divorced	high.school	no	yes	no	cellular	

5 rows × 21 columns

In [119…

```
# Eliminar registros inconsistentes

df = df[~((df["duration"] == 0) & (df["y"] == "yes"))]
```

```
print("Nueva dimensión del dataset:", df.shape)
```

```
Nueva dimensión del dataset: (41351, 21)
```

Validación de consistencia de llamadas

Identificamos 25 registros donde la duración de la llamada es igual a cero (`duration = 0`) pero el cliente aparece como aceptando el depósito a plazo (`y = yes`).

Esta combinación representa una inconsistencia ya que un cliente no podría aceptar el producto sin haber tenido interacción con las llamadas. Dado que estos registros representan una proporción muy pequeña del dataset, se decidió eliminarlos para mantener la coherencia del análisis.

Interpretación de estadísticas descriptivas

El análisis descriptivo muestra que la edad promedio de los clientes es aproximadamente 40 años, con un rango que va desde 17 - 98 años, lo que indica que la campaña es multitarget. La duración promedio de las llamadas es cercana a 257 segundos, aunque existe una alta dispersión con llamadas que alcanzan valores mucho más altos. Esto sugiere que algunas llamadas requieren mucho más tiempo para explicar los productos o convencer a los clientes.

La variable `campaign` muestra que algunos clientes han sido contactados varias veces durante la campaña llegando hasta 50 veces en casos extremos.

In [120…

```
df.describe()
```

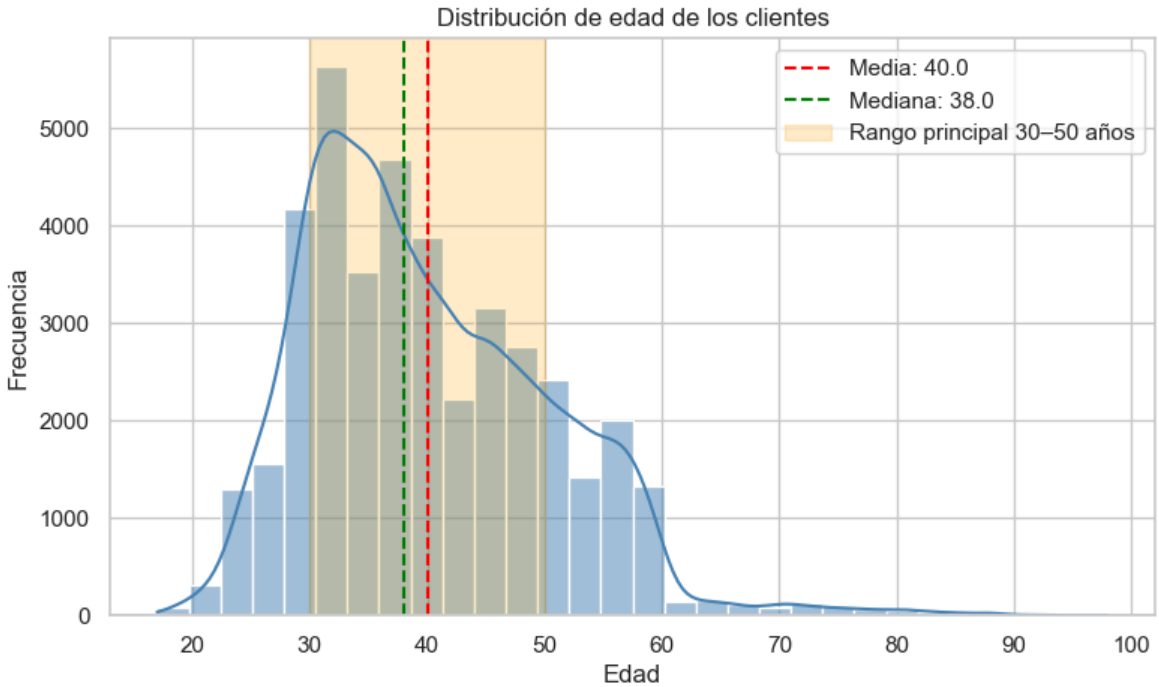
Out[120…

	age	duration	campaign	pdays	previous	e
count	41351.000000	41351.000000	41351.000000	41351.000000	41351.000000	41
mean	40.024933	257.222606	2.570434	962.547169	0.172765	
std	10.416508	259.298837	2.776158	186.734891	0.494511	
min	17.000000	0.000000	1.000000	0.000000	0.000000	
25%	32.000000	101.000000	1.000000	999.000000	0.000000	
50%	38.000000	179.000000	2.000000	999.000000	0.000000	
75%	47.000000	318.000000	3.000000	999.000000	0.000000	
max	98.000000	4918.000000	56.000000	999.000000	7.000000	

In [121…

```
import matplotlib.pyplot as plt
import seaborn as sns

media = df["age"].mean()
mediana = df["age"].median()

plt.figure(figsize=(9,5))
```

```
sns.histplot(
    df["age"],
    bins=30,
    kde=True,
    color="steelblue"
)

# media
plt.axvline(media, color="red", linestyle="--", label=f"Media: {media:.1f

# mediana
plt.axvline(mediana, color="green", linestyle="--", label=f"Mediana: {med

# resaltar rango 30-50
plt.axvspan(30, 50, alpha=0.2, color="orange", label="Rango principal 30-

plt.title("Distribución de edad de los clientes")
plt.xlabel("Edad")
plt.ylabel("Frecuencia")

plt.legend()

plt.show()
```

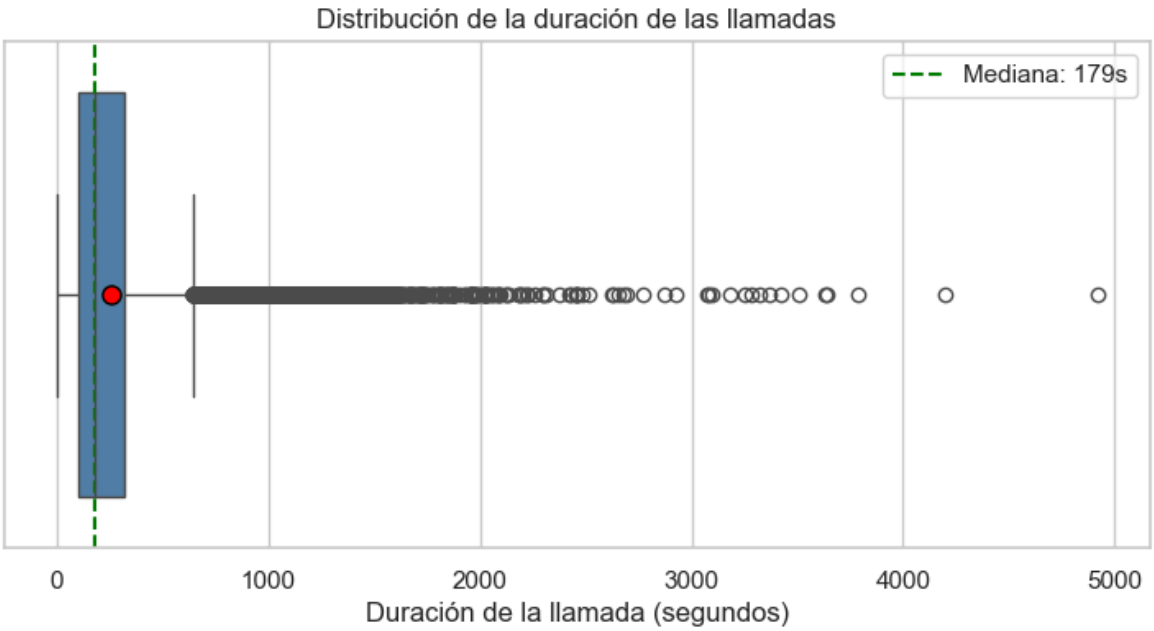


El gráfico nos muestra que la mayor concentración de clientes se encuentra entre los 30 y 50 años, lo que sugiere que este grupo de personas representan una parte importante de la BD contactada.

In [122…

```
import matplotlib.pyplot as plt
import seaborn as sns

media = df["duration"].mean()
mediana = df["duration"].median()

plt.figure(figsize=(9,4))
```

```
sns.boxplot(
    x=df["duration"],
    color="steelblue",
    showmeans=True,
    meanprops={
        "marker":"o",
        "markerfacecolor":"red",
        "markeredgecolor":"black",
        "markersize":"8"
    }
)

# línea mediana
plt.axvline(mediana, color="green", linestyle="--", label=f"Mediana: {med

plt.title("Distribución de la duración de las llamadas")
plt.xlabel("Duración de la llamada (segundos)")

plt.legend()

plt.show()
```



In [ ]:

Análisis de la duración de las llamadas

El boxplot nos muestra que la mayoría de las llamadas tiene duraciones bajas, aunque existen varios valores atípicos con llamadas considerablemente más largas. Esto sugiere que algunas llamadas tuvieron más tiempo de conversación, posiblemente por mayor interés del cliente o mayor explicación del producto.

Conclusión de calidad de datos

En la revisión inicial del dataset se analizaron duplicados, valores nulos, tipos de datos y consistencia de observaciones. Los valores faltantes en este caso no necesariamente aparecen como nulos, sino que en varias variables categóricas están

codificados como `"unknown"` por lo que deberían ser tratados durante el análisis. Además validamos la coherencia de variables numéricas relevantes como edad, duración y número de contactos.

Parte 2: Feature Transformation

Aquí crearemos nuevas variables a partir de las variables originales del dataset para poder facilitar el análisis y generar nuevas perspectivas sobre el comportamiento de los clientes frente a la campaña.

Creamos una variable binaria (`y_bin`) para representar la variable objetivo de forma numérica, donde:

- 1 = el cliente aceptó el depósito a plazo
- 0 = el cliente no aceptó nuestra campaña

In [123…
```
df["y_bin"] = df["y"].map({"yes":1, "no":0})

df["y_bin"].value_counts()
```

Out[123…
```
y_bin
0    36712
1     4639
Name: count, dtype: int64
```

Creamos una variable categórica que agrupa a los clientes por rangos de edad. Así analizamos si ciertos segmentos de edad presentan mayor probabilidad de aceptar la campaña.

In [124…
```
df["grupo_edad"] = pd.cut(
    df["age"],
    bins=[0,30,40,50,60,100],
    labels=["<30","30-40","40-50","50-60","60+"]
)

df["grupo_edad"].value_counts()
```

Out[124…
```
grupo_edad
30-40    16451
40-50    10288
<30       7405
50-60     6296
60+        911
Name: count, dtype: int64
```

Creamos una variable que identifice a los clientes que fueron contactados más de tres veces durante la campaña actual. Esto nos permite analizar si un mayor número de contactos aumenta o reduce la probabilidad de conversión.

In [125…
```
df["muchos_contactos"] = (df["campaign"] > 3).astype(int)

df["muchos_contactos"].value_counts()
```

Out[125…
```
muchos_contactos
0    33676
1     7675
Name: count, dtype: int64
```

Creamos una variable que nos muestre si el cliente ha sido contactado en campañas anteriores. Así analizamos si la experiencia previa con campañas del banco va a influir en la decisión de aceptar el producto.

In [126…
```
df["contactado_previamente"] = (df["previous"] > 0).astype(int)

df["contactado_previamente"].value_counts()
```

Out[126…
```
contactado_previamente
0    35708
1     5643
Name: count, dtype: int64
```

Creamos una variable para identificar clientes que nunca han sido contactados por campañas del banco. Así analizamos si los clientes nuevos responden de una manera diferente a la campaña.

In [127…
```
df["cliente_nuevo"] = (df["pdays"] == 999).astype(int)

df["cliente_nuevo"].value_counts()
```

Out[127…
```
cliente_nuevo
1    39833
0     1518
Name: count, dtype: int64
```

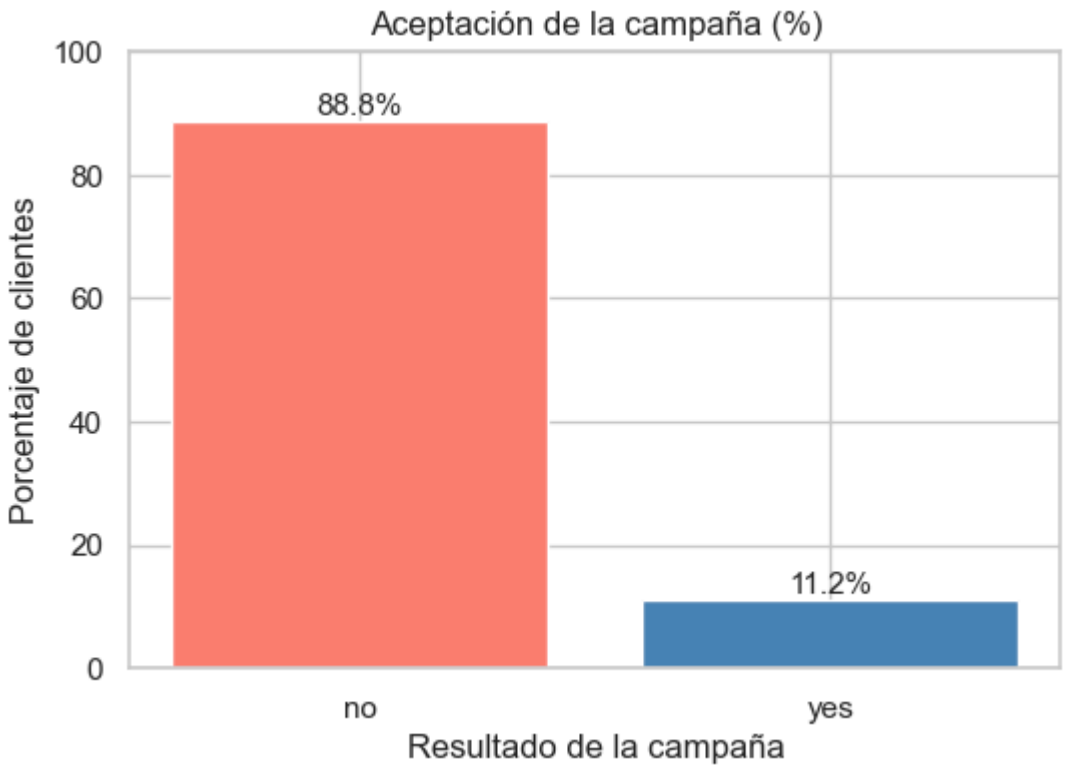
In [128…
```
# Distribución porcentual de variables creadas

print("Distribución de la variable objetivo (y_bin):")
print((df["y_bin"].value_counts(normalize=True)*100).round(2))

print("\nDistribución de grupos de edad:")
print((df["grupo_edad"].value_counts(normalize=True)*100).round(2))

print("\nClientes con muchos contactos:")
print((df["muchos_contactos"].value_counts(normalize=True)*100).round(2))

print("\nClientes contactados previamente:")
print((df["contactado_previamente"].value_counts(normalize=True)*100).rou

print("\nClientes nuevos:")
print((df["cliente_nuevo"].value_counts(normalize=True)*100).round(2))
```

```
Distribución de la variable objetivo (y_bin):
y_bin
0    88.78
1    11.22
Name: proportion, dtype: float64

Distribución de grupos de edad:
grupo_edad
30-40    39.78
40-50    24.88
<30      17.91
50-60    15.23
60+       2.20
Name: proportion, dtype: float64

Clientes con muchos contactos:
muchos_contactos
0    81.44
1    18.56
Name: proportion, dtype: float64

Clientes contactados previamente:
contactado_previamente
0    86.35
1    13.65
Name: proportion, dtype: float64

Clientes nuevos:
cliente_nuevo
1    96.33
0     3.67
Name: proportion, dtype: float64
```

In [129…

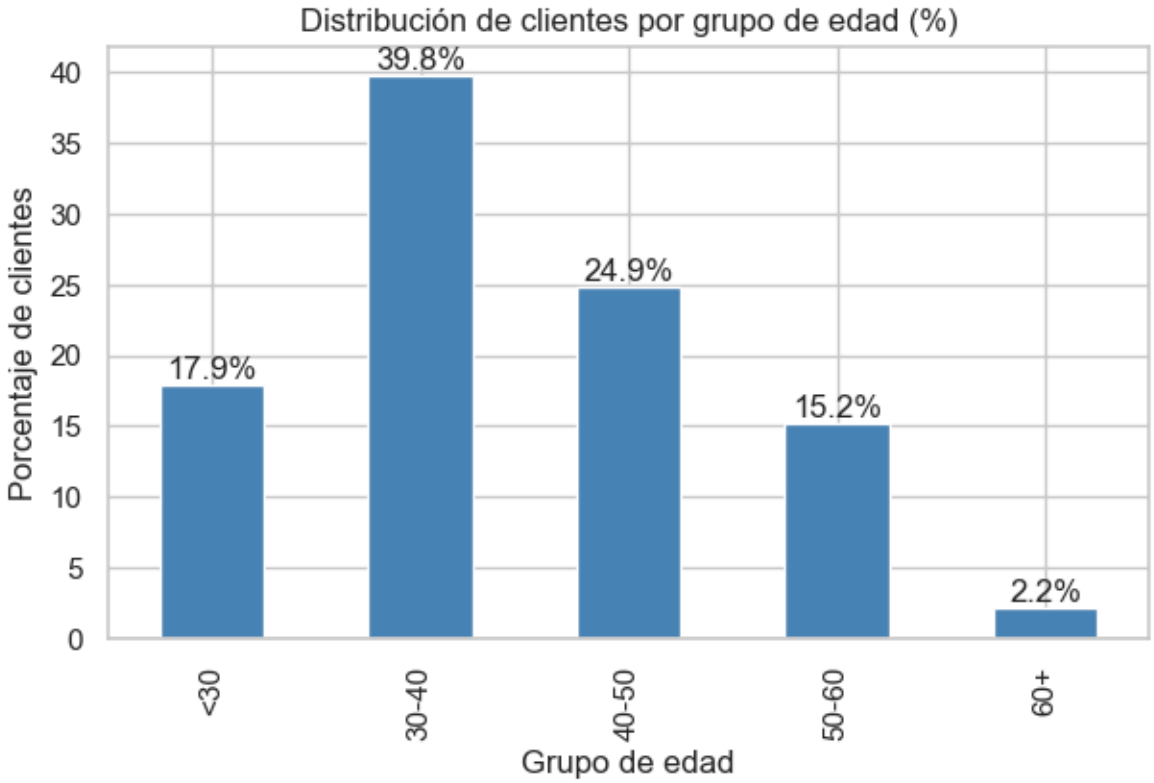
```python
# calcular porcentajes
aceptacion = df["y"].value_counts(normalize=True).sort_index() * 100

plt.figure(figsize=(6,4))

bars = plt.bar(
    aceptacion.index,
    aceptacion.values,
    color=["salmon","steelblue"]
)

plt.title("Aceptación de la campaña (%)", fontsize=12)
plt.xlabel("Resultado de la campaña")
plt.ylabel("Porcentaje de clientes")

# límite del eje
plt.ylim(0,100)

# etiquetas de porcentaje
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        height + 1,
        f"{height:.1f}%",
        ha="center",
        fontsize=11
```

```
    )

plt.show()
```



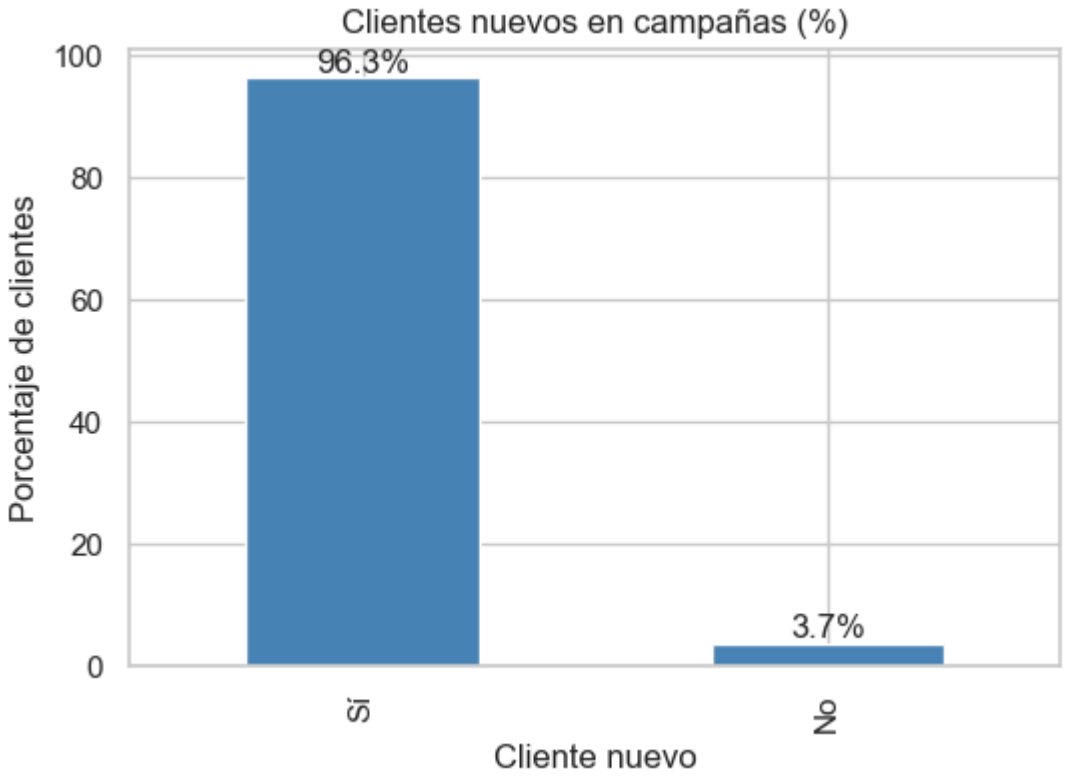
In [130…

```
plt.figure(figsize=(7,4))

ax = (df["grupo_edad"].value_counts(normalize=True)*100)\
        .sort_index()\
        .plot(kind="bar", color="steelblue")

plt.title("Distribución de clientes por grupo de edad (%)")
plt.ylabel("Porcentaje de clientes")
plt.xlabel("Grupo de edad")

for p in ax.patches:
    ax.annotate(f'{p.get_height():.1f}%',
                (p.get_x() + p.get_width()/2., p.get_height()),
                ha='center', va='bottom')

plt.show()
```



In [131…

```
cliente_nuevo_labels = df["cliente_nuevo"].map({1: "Sí", 0: "No"})

plt.figure(figsize=(6,4))
ax = (cliente_nuevo_labels.value_counts(normalize=True)*100)\
        .plot(kind="bar", color="steelblue")

plt.title("Clientes nuevos en campañas (%)")
plt.ylabel("Porcentaje de clientes")
plt.xlabel("Cliente nuevo")

for p in ax.patches:
    ax.annotate(f'{p.get_height():.1f}%',
                (p.get_x() + p.get_width()/2., p.get_height()),
                ha='center', va='bottom')

plt.show()
```



Resumen de variables creadas

Las variables generadas nos van a permitir segmentar a nuestros clientes según edad, experiencia previa con campañas del banco y las veces que han sido contactados durante la campaña actual.

Entonces a partir de estas transformaciones vemos que aproximadamente el **88.7% de los clientes no aceptó la campaña**, mientras que solo **11.3% decidió suscribirse al depósito a plazo**.

También el **65% de los clientes está entre 30 y 50 años**, lo que muestra que este segmento representa una parte importante de la base de clientes contactada por el banco.

Finalmente, se observa que alrededor del **96% de los clientes no había sido contactado previamente en campañas anteriores**, lo que nos hace pensar que gran parte de la campaña se dirigió a clientes nuevos dentro del proceso de marketing telefónico.

Parte 3: Análisis correlacional

Analizaremos la relación entre las variables numéricas del dataset utilizando el coeficiente de correlación de Pearson.

El objetivo es identificar qué variables presentan relaciones entre sí y analizar si dichas relaciones tienen sentido desde una perspectiva de negocio.

In [132…
```
variables_numericas = df.select_dtypes(include=["int64","float64"])

variables_numericas.columns
```

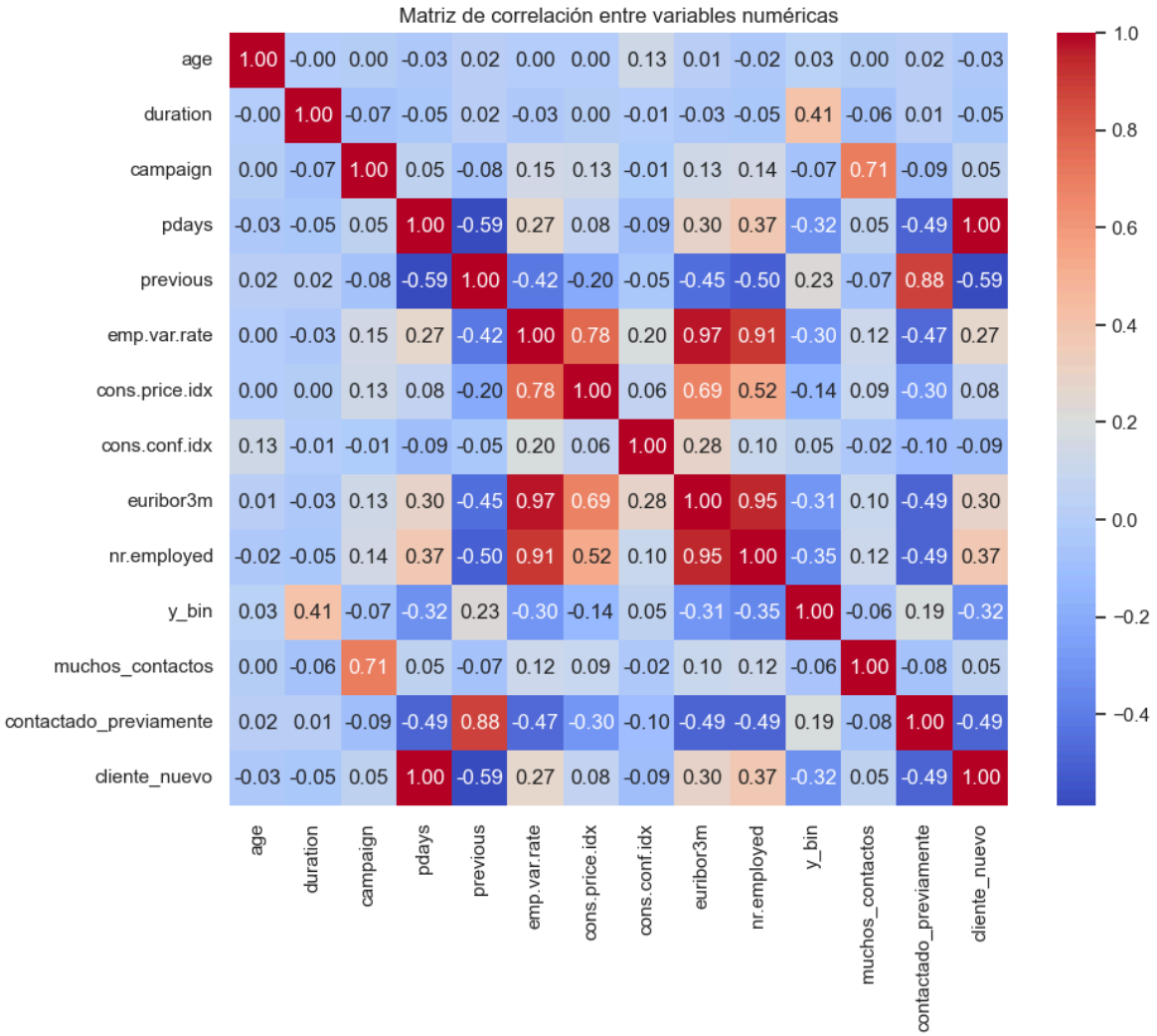
Out[132…
```
Index(['age', 'duration', 'campaign', 'pdays', 'previous', 'emp.var.rat
e',
       'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed', 'y
_bin',
       'muchos_contactos', 'contactado_previamente', 'cliente_nuevo'],
      dtype='str')
```

In [133…
```
correlacion = variables_numericas.corr()

correlacion
```

Out[133…

	age	duration	campaign	pdays	previous
age	1.000000	-0.000913	0.004710	-0.033905	0.023953
duration	-0.000913	1.000000	-0.072225	-0.047845	0.021031
campaign	0.004710	-0.072225	1.000000	0.052728	-0.079098
pdays	-0.033905	-0.047845	0.052728	1.000000	-0.587458
previous	0.023953	0.021031	-0.079098	-0.587458	1.000000
emp.var.rate	0.000031	-0.028533	0.150594	0.270862	-0.420631
cons.price.idx	0.001390	0.004831	0.127472	0.079011	-0.203568
cons.conf.idx	0.128856	-0.008308	-0.013890	-0.091345	-0.051396
euribor3m	0.011155	-0.033467	0.134939	0.296632	-0.454602
nr.employed	-0.017364	-0.045267	0.144032	0.372376	-0.501315
y_bin	0.030282	0.405839	-0.066390	-0.324680	0.230042
muchos_contactos	0.004471	-0.063310	0.706814	0.054180	-0.069933
contactado_previamente	0.016064	0.011987	-0.087495	-0.491065	0.878846
cliente_nuevo	-0.033830	-0.047903	0.052713	0.999992	-0.587407

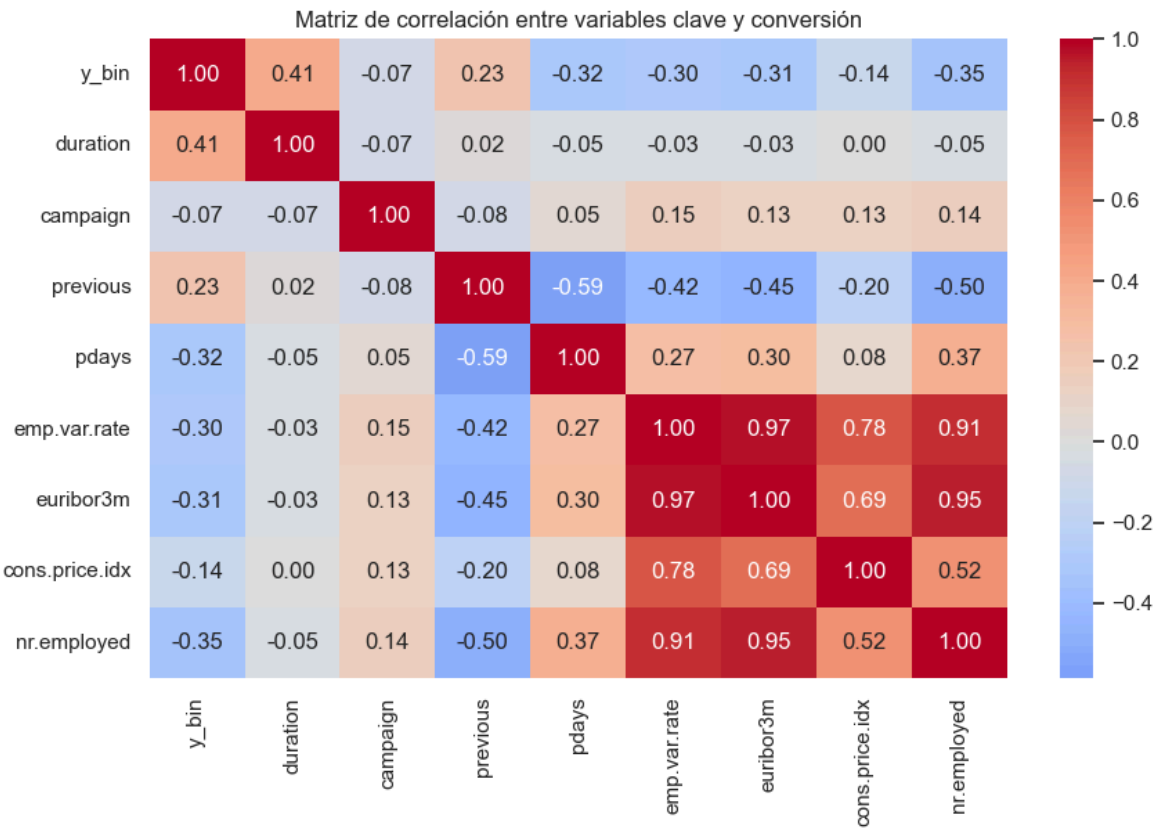
In [134…
```
plt.figure(figsize=(10,8))

sns.heatmap(
    correlacion,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Matriz de correlación entre variables numéricas")
plt.show()
```



In [135…

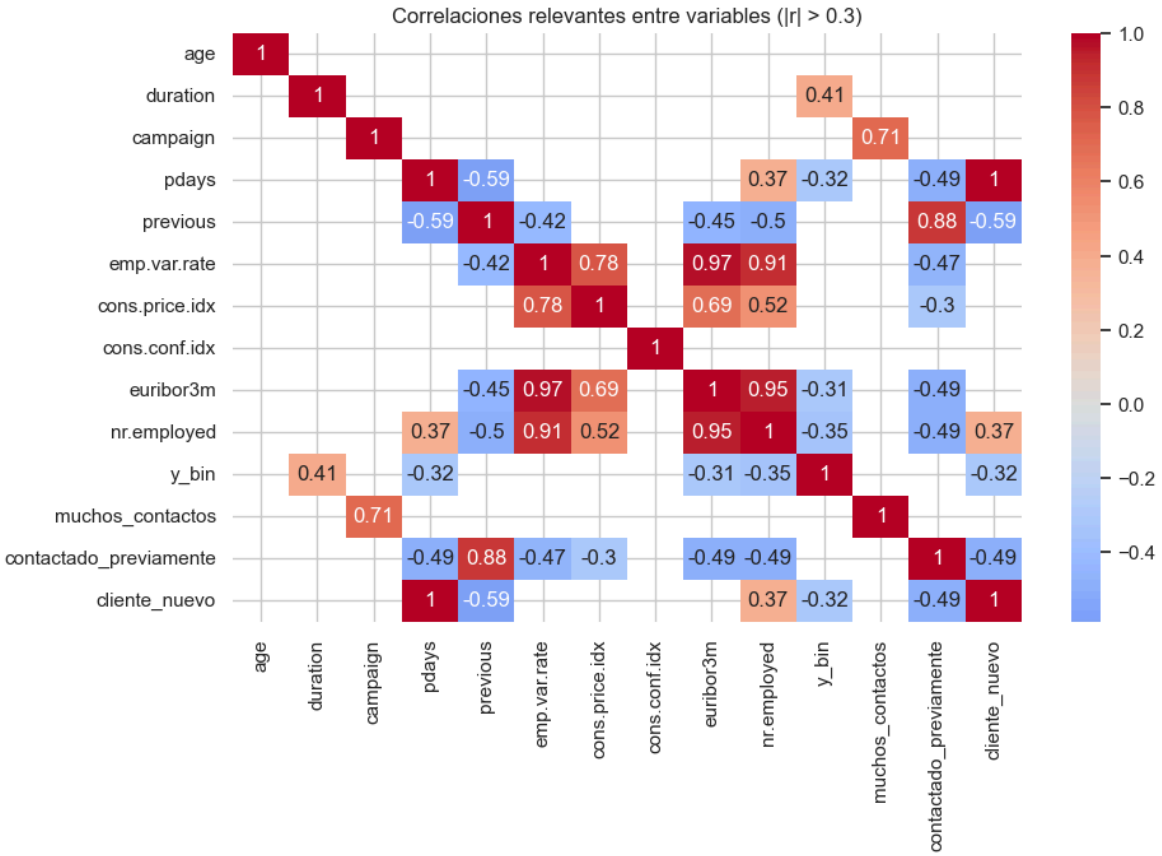
```
import seaborn as sns
import matplotlib.pyplot as plt

# variables clave incluyendo conversión
variables_clave = [
    "y_bin",            # resultado de campaña (0=no, 1=si)
    "duration",
    "campaign",
    "previous",
    "pdays",
    "emp.var.rate",
    "euribor3m",
    "cons.price.idx",
    "nr.employed"
]

# calcular correlación
corr = df[variables_clave].corr()

# gráfico
plt.figure(figsize=(10,6))

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    center=0,
    fmt=".2f"
```

```
)

plt.title("Matriz de correlación entre variables clave y conversión")

plt.show()
```



In [136…

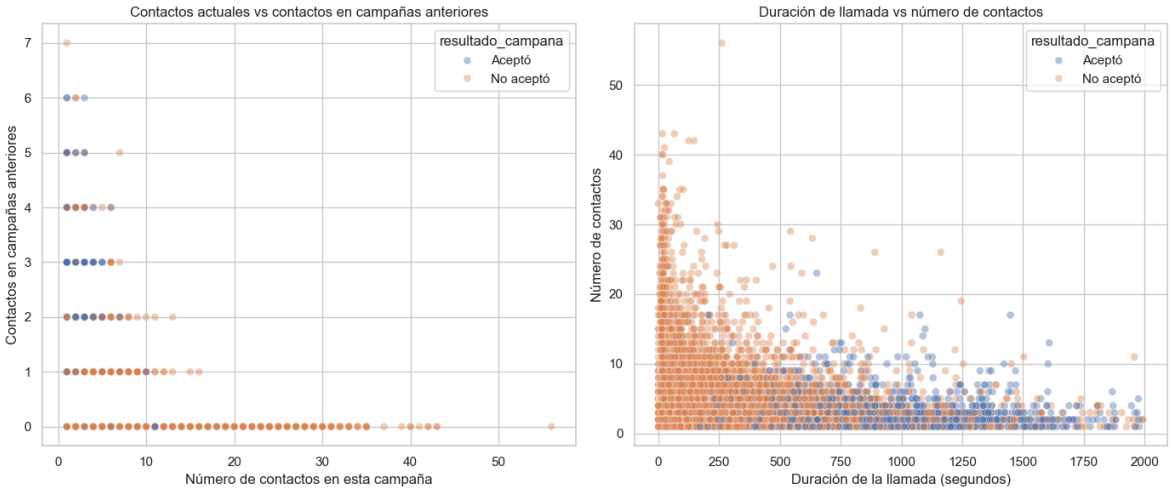
```
import seaborn as sns
import matplotlib.pyplot as plt

corr = df.corr(numeric_only=True)

# filtrar correlaciones fuertes
corr_filtrado = corr[(corr > 0.3) | (corr < -0.3)]

plt.figure(figsize=(10,6))

sns.heatmap(
    corr_filtrado,
    annot=True,
    cmap="coolwarm",
    center=0
)

plt.title("Correlaciones relevantes entre variables (|r| > 0.3)")
plt.show()
```



In [140…

```
import matplotlib.pyplot as plt
import seaborn as sns

# Estilo visual
sns.set(style="whitegrid")

# Crear variable para la leyenda del gráfico
df["resultado_campana"] = df["y"].map({
    "yes": "Aceptó",
    "no": "No aceptó"
})

# Filtrar outliers de duración (recomendado)
df_plot = df[df["duration"] < 2000]

# Crear figura
fig, axes = plt.subplots(1, 2, figsize=(14,6))

# ------------------------------------------
# Gráfico 1: campaign vs previous
# ------------------------------------------
sns.scatterplot(
    data=df_plot,
    x="campaign",
    y="previous",
    hue="resultado_campana",
    alpha=0.4,
    s=40,
    ax=axes[0]
)

axes[0].set_title("Contactos actuales vs contactos en campañas anteriores
axes[0].set_xlabel("Número de contactos en esta campaña")
```

```
axes[0].set_ylabel("Contactos en campañas anteriores")

# ------------------------------------------
# Gráfico 2: duration vs campaign
# ------------------------------------------
sns.scatterplot(
    data=df_plot,
    x="duration",
    y="campaign",
    hue="resultado_campana",
    alpha=0.4,
    s=40,
    ax=axes[1]
)

axes[1].set_title("Duración de llamada vs número de contactos")
axes[1].set_xlabel("Duración de la llamada (segundos)")
axes[1].set_ylabel("Número de contactos")

# Ajustar layout
plt.tight_layout()

# Mostrar gráficos
plt.show()
```



Identificación de correlaciones relevantes

El análisis de correlación permite identificar variables que presentan relaciones fuertes entre sí.

Se observan correlaciones altas entre algunas variables económicas como `euribor3m`, `emp.var.rate` y `nr.employed`. Esto tiene sentido desde una perspectiva económica, ya que todas representan indicadores macroeconómicos relacionados con el mercado laboral y las tasas de interés.

También se observan correlaciones fuertes entre variables creadas durante el proceso de transformación de variables. Por ejemplo, `campaign` y `muchos_contactos` presentan una correlación alta debido a que la segunda variable se construyó a partir de la primera.

De forma similar, `previous` presenta una correlación fuerte con `contactado_previamente`, lo cual es esperable porque esta última variable indica si el número de contactos previos es mayor que cero.

Finalmente, `pdays` y `cliente_nuevo` presentan una correlación casi perfecta, ya que la variable `cliente_nuevo` se construyó a partir del valor especial 999 presente en `pdays`.

In [ ]:
```
# Identificar correlaciones fuertes

corr_matrix = df.corr(numeric_only=True)

# Convertir matriz a formato tabla
correlaciones_fuertes = (
    corr_matrix
    .abs()
    .stack()
    .reset_index()
)

correlaciones_fuertes.columns = ["Variable 1", "Variable 2", "Correlación

# eliminar correlación de variable consigo misma
correlaciones_fuertes = correlaciones_fuertes[
    correlaciones_fuertes["Variable 1"] != correlaciones_fuertes["Variabl
]

# filtrar correlaciones mayores a 0.7
correlaciones_fuertes = correlaciones_fuertes[
    correlaciones_fuertes["Correlación"] > 0.7
]

# ordenar de mayor a menor
correlaciones_fuertes = correlaciones_fuertes.sort_values(
    by="Correlación", ascending=False
)

print(correlaciones_fuertes)
```

```
                Variable 1              Variable 2  Correlación
55                   pdays           cliente_nuevo     0.999992
185          cliente_nuevo                   pdays     0.999992
78            emp.var.rate               euribor3m     0.972273
117              euribor3m            emp.var.rate     0.972273
121              euribor3m             nr.employed     0.945157
134            nr.employed               euribor3m     0.945157
79            emp.var.rate             nr.employed     0.907057
131            nr.employed            emp.var.rate     0.907057
68                 previous  contactado_previamente     0.878846
172  contactado_previamente                 previous     0.878846
76            emp.var.rate          cons.price.idx     0.775439
89          cons.price.idx            emp.var.rate     0.775439
39                campaign        muchos_contactos     0.706814
156        muchos_contactos                campaign     0.706814
```

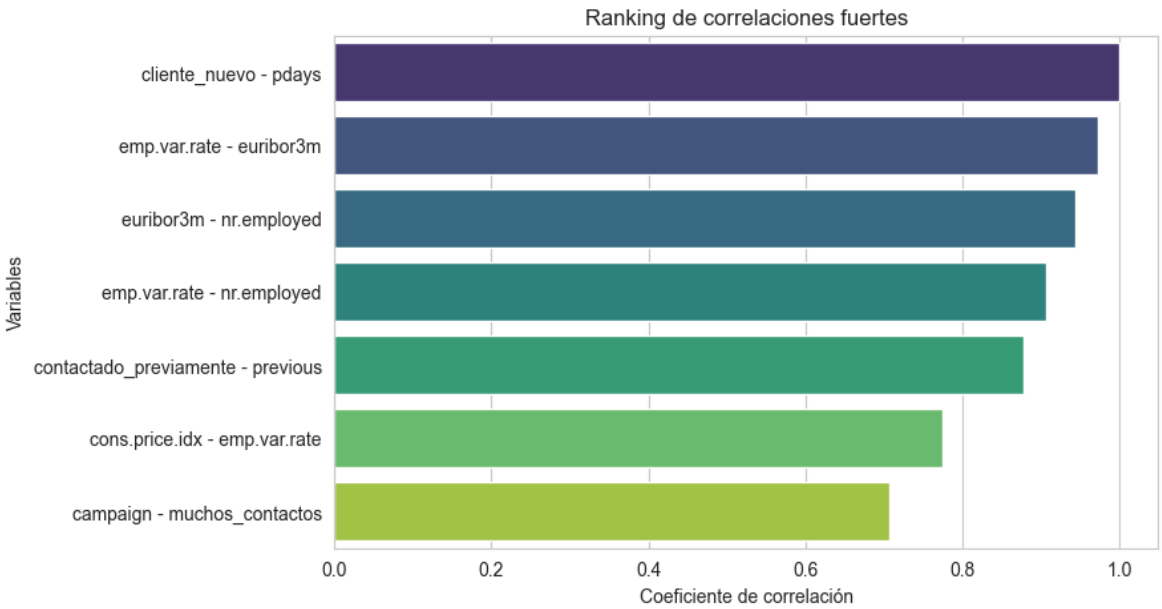
In [ ]:
```
# eliminar duplicados
correlaciones_fuertes = correlaciones_fuertes[
    correlaciones_fuertes["Variable 1"] < correlaciones_fuertes["Variable
```

```
]

# ordenar de mayor a menor
correlaciones_fuertes = correlaciones_fuertes.sort_values(
    by="Correlación", ascending=False
)

print(correlaciones_fuertes)
```

```
                Variable 1        Variable 2  Correlación
185           cliente_nuevo             pdays     0.999992
78             emp.var.rate         euribor3m     0.972273
121               euribor3m       nr.employed     0.945157
79             emp.var.rate       nr.employed     0.907057
172  contactado_previamente          previous     0.878846
89           cons.price.idx      emp.var.rate     0.775439
39                 campaign  muchos_contactos     0.706814
```

In [ ]:
```
correlaciones_fuertes["Variables"] = (
    correlaciones_fuertes["Variable 1"] + " - " + correlaciones_fuertes["
)
```

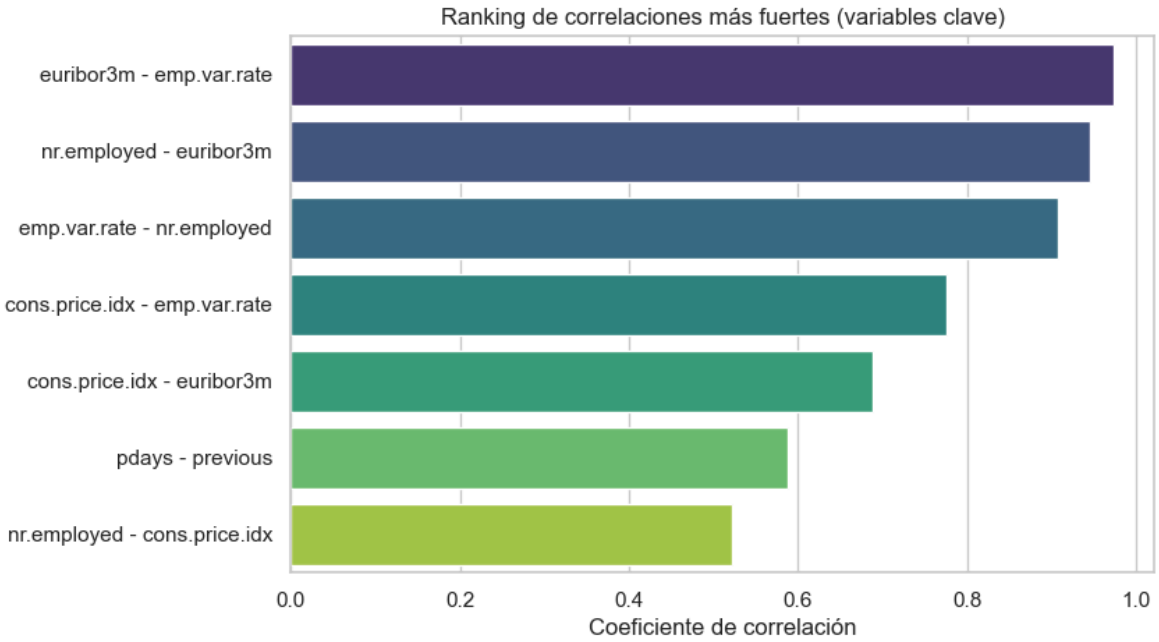
In [ ]:
```
plt.figure(figsize=(8,5))

sns.barplot(
    x="Correlación",
    y="Variables",
    data=correlaciones_fuertes,
    hue="Variables",
    legend=False,
    palette="viridis"
)

plt.title("Ranking de correlaciones fuertes")
plt.xlabel("Coeficiente de correlación")
plt.ylabel("Variables")

plt.show()
```



In [ ]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

```python
import numpy as np
import pandas as pd

variables_clave = [
    "y_bin",
    "duration",
    "campaign",
    "previous",
    "pdays",
    "emp.var.rate",
    "euribor3m",
    "cons.price.idx",
    "nr.employed"
]

# matriz de correlación
corr = df[variables_clave].corr()

# convertir a tabla de correlaciones
corr_pairs = corr.abs().unstack()

# eliminar autocorrelaciones
corr_pairs = corr_pairs[corr_pairs < 1]

# ordenar
corr_pairs = corr_pairs.sort_values(ascending=False)

# eliminar duplicados
corr_pairs = corr_pairs.drop_duplicates()

# top correlaciones
top_corr = corr_pairs.head(7)

# convertir a dataframe
top_corr = top_corr.reset_index()
top_corr.columns = ["Variable 1", "Variable 2", "Correlación"]

# crear etiqueta
top_corr["par"] = top_corr["Variable 1"] + " - " + top_corr["Variable 2"]

plt.figure(figsize=(8,5))

sns.barplot(
    data=top_corr,
    x="Correlación",
    y="par",
    palette="viridis"
)

plt.title("Ranking de correlaciones más fuertes (variables clave)")
plt.xlabel("Coeficiente de correlación")
plt.ylabel("")

plt.show()
```

```
/var/folders/6h/p2w9qddx31dd5ywkxhs8s3f40000gn/T/ipykernel_6877/403190595
0.py:45: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(
```



Visualización de correlaciones fuertes

Para facilitar la interpretación del análisis correlacional se visualizan las correlaciones más altas identificadas en el dataset.

Este gráfico permite observar de manera más clara qué pares de variables presentan relaciones más fuertes. Como se observa, varias de las correlaciones más altas corresponden a variables derivadas durante el proceso de transformación de datos o a indicadores macroeconómicos relacionados entre sí.

Interpretación del análisis correlacional

A partir de la matriz de correlación se identificaron algunas relaciones relevantes entre variables del dataset.

Primero, observamos correlaciones muy altas entre variables macroeconómicas como **euribor3m, emp.var.rate y nr.employed**. Estas variables representan indicadores del entorno económico, por lo que es esperable que se muevan de manera conjunta.

También se identifican correlaciones fuertes entre variables derivadas durante el proceso de transformación. Por ejemplo, **campaign y muchos_contactos** presentan una correlación alta, lo cual es lógico ya que la variable *muchos_contactos* se construyó a partir del número de contactos realizados durante la campaña.

De forma similar, **previous y contactado_previamente** muestran una correlación elevada, ya que ambas variables reflejan el historial de contactos con el cliente en campañas anteriores.

Asimismo, **pdays y cliente_nuevo** presentan una correlación casi perfecta, debido a que la variable *cliente_nuevo* se construyó utilizando el valor especial 999 presente en *pdays* para identificar clientes que no habían sido contactados anteriormente.

Finalmente, al analizar la correlación con la variable objetivo (*y_bin*), se observa que la **duración de la llamada (duration)** presenta la relación positiva más fuerte con la aceptación del producto. Esto sugiere que llamadas más largas suelen estar asociadas con una mayor probabilidad de conversión.

En contraste, variables relacionadas con el entorno macroeconómico muestran correlaciones negativas moderadas, lo que podría reflejar que el contexto económico influye en la decisión de los clientes de aceptar el depósito a plazo.

En general, los resultados del análisis correlacional son coherentes con la lógica del negocio y permiten identificar variables potencialmente relevantes para futuros modelos predictivos.

Parte 4: Análisis descriptivo

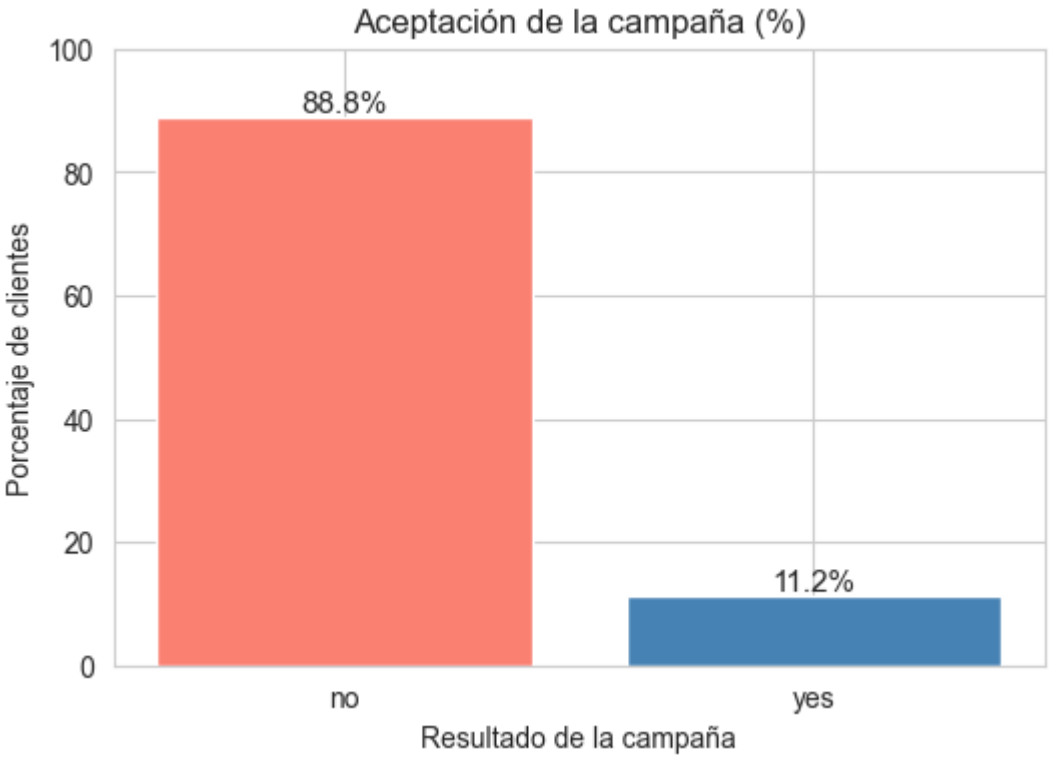
Aquí exploramos visualmente algunas relaciones relevantes entre las variables del dataset con el objetivo de comprender mejor el comportamiento de los clientes frente a la campaña de marketing.

A diferencia del análisis correlacional, que se enfoca en medir la relación estadística entre variables, el análisis descriptivo busca identificar patrones visuales que ayuden a interpretar el comportamiento de los clientes y generar insights desde una perspectiva de negocio.

In [ ]:
```
# Variable descriptiva para el análisis bivariado
df["resultado_campana"] = df["y_bin"].map({
    1: "Aceptó",
    0: "No aceptó"
})
```

In [ ]:
```
# calcular porcentajes
aceptacion = df["y"].value_counts(normalize=True).sort_index() * 100

plt.figure(figsize=(6,4))

bars = plt.bar(
    aceptacion.index,
    aceptacion.values,
    color=["salmon","steelblue"]
)

plt.title("Aceptación de la campaña (%)", fontsize=12)
plt.xlabel("Resultado de la campaña")
```

```
plt.ylabel("Porcentaje de clientes")

# límite del eje
plt.ylim(0,100)

# etiquetas de porcentaje
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        height + 1,
        f"{height:.1f}%",
        ha="center",
        fontsize=11
    )

plt.show()
```



Distribución de aceptación de la campaña

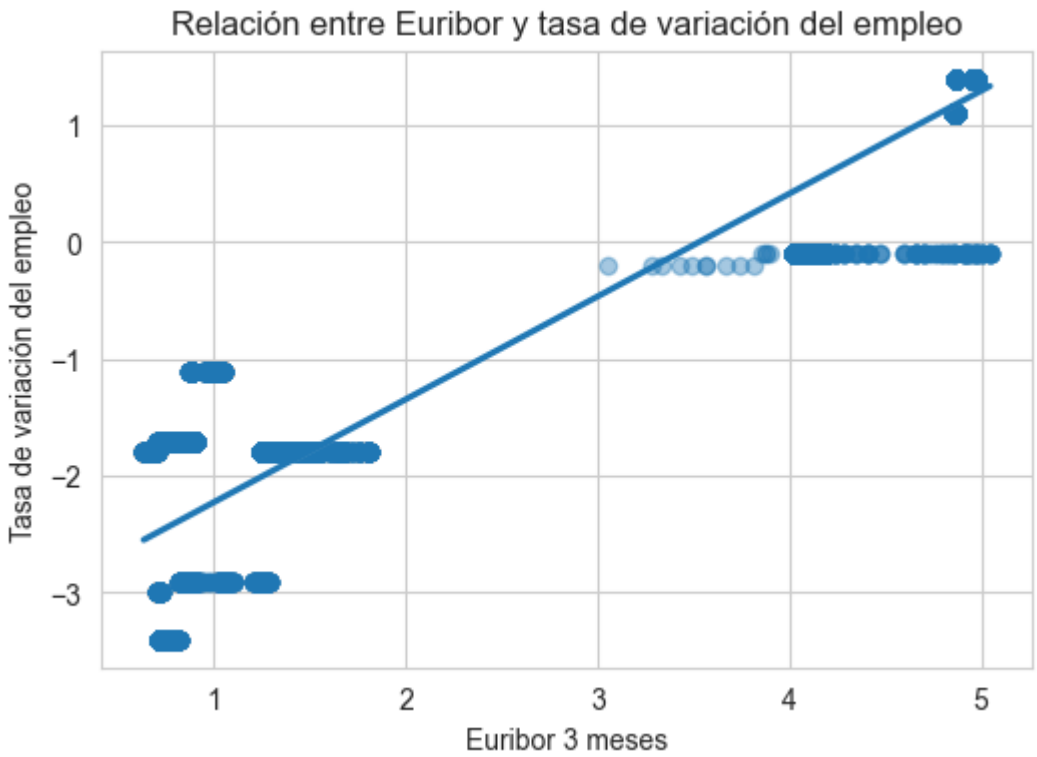
Antes de analizar qué variables influyen en la aceptación del depósito a plazo, volvemos a presentar la distribución general del resultado de la campaña.

El gráfico muestra que aproximadamente el 11% de los clientes aceptaron la oferta, mientras que cerca del 89% no lo hicieron. Este resultado es común en campañas de marketing directo, donde las tasas de conversión suelen ser relativamente bajas.

Por esta razón, resulta especialmente importante analizar qué características de los clientes están asociadas con una mayor probabilidad de aceptación del producto.

In [ ]:
```
plt.figure(figsize=(6,4))

sns.regplot(
```

```
    x=df["euribor3m"],
    y=df["emp.var.rate"],
    scatter_kws={"alpha":0.4}
)

plt.title("Relación entre Euribor y tasa de variación del empleo")
plt.xlabel("Euribor 3 meses")
plt.ylabel("Tasa de variación del empleo")

plt.show()
```



Relación entre indicadores macroeconómicos

El gráfico muestra la relación entre la tasa Euribor a 3 meses y la tasa de variación del empleo.

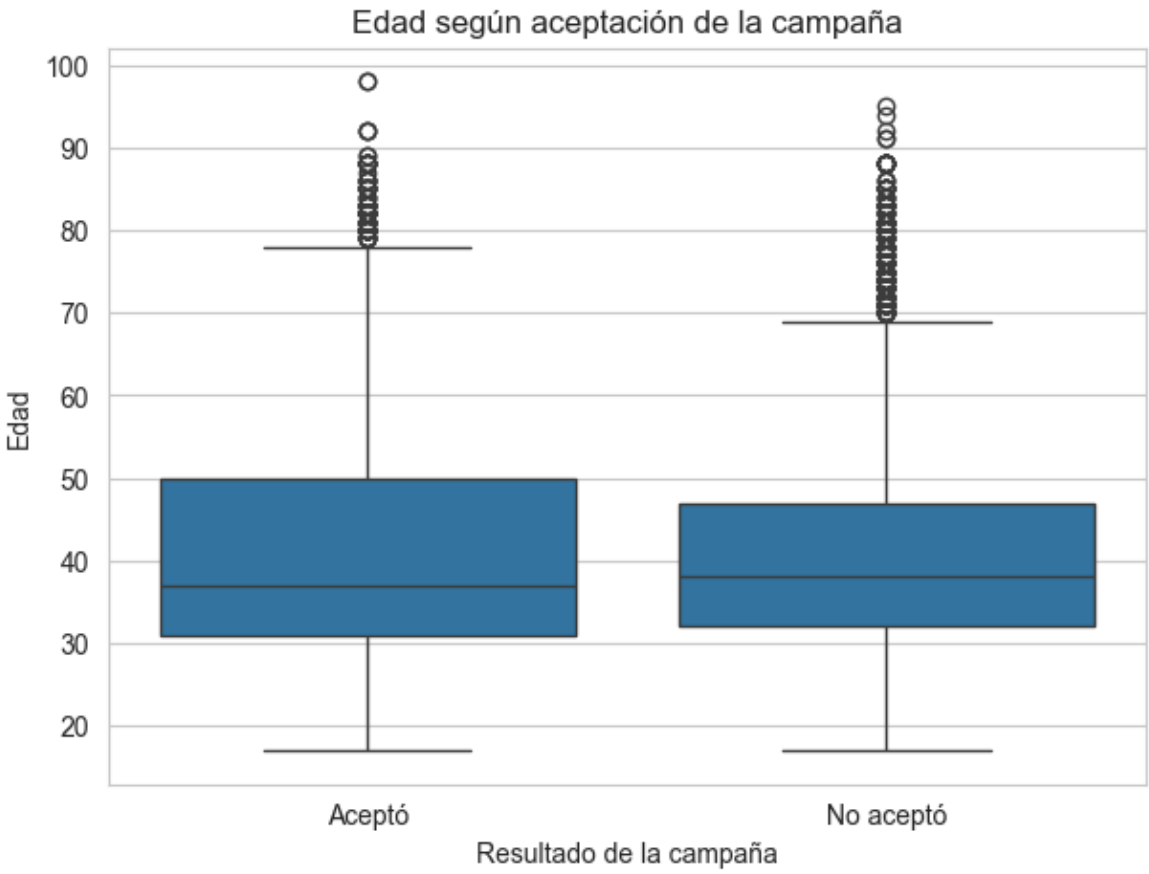
Se observa una relación positiva clara entre ambas variables, lo que confirma la fuerte correlación identificada previamente en el análisis correlacional. Esto tiene sentido desde una perspectiva económica, ya que ambas variables reflejan condiciones del entorno macroeconómico.

Pregunta 1: ¿La edad influye en la aceptación de la campaña?

Para responder esta pregunta se analiza la relación entre la edad de los clientes y el resultado de la campaña de marketing. El objetivo es evaluar si existen diferencias en la distribución de edades entre los clientes que aceptaron el depósito a plazo y aquellos que no lo hicieron.

In [ ]:
```
plt.figure(figsize=(7,5))

sns.boxplot(
```

```
    x="resultado_campana",
    y="age",
    data=df
)

plt.title("Edad según aceptación de la campaña")
plt.xlabel("Resultado de la campaña")
plt.ylabel("Edad")

plt.show()
```



Edad y aceptación de la campaña

El análisis de la distribución de edades no muestra diferencias claras entre los clientes que aceptaron la campaña y aquellos que no lo hicieron. Las medianas y los rangos intercuartílicos de ambos grupos son bastante similares, lo que sugiere que la edad, por sí sola, no parece ser un factor determinante en la decisión de aceptar el producto financiero.

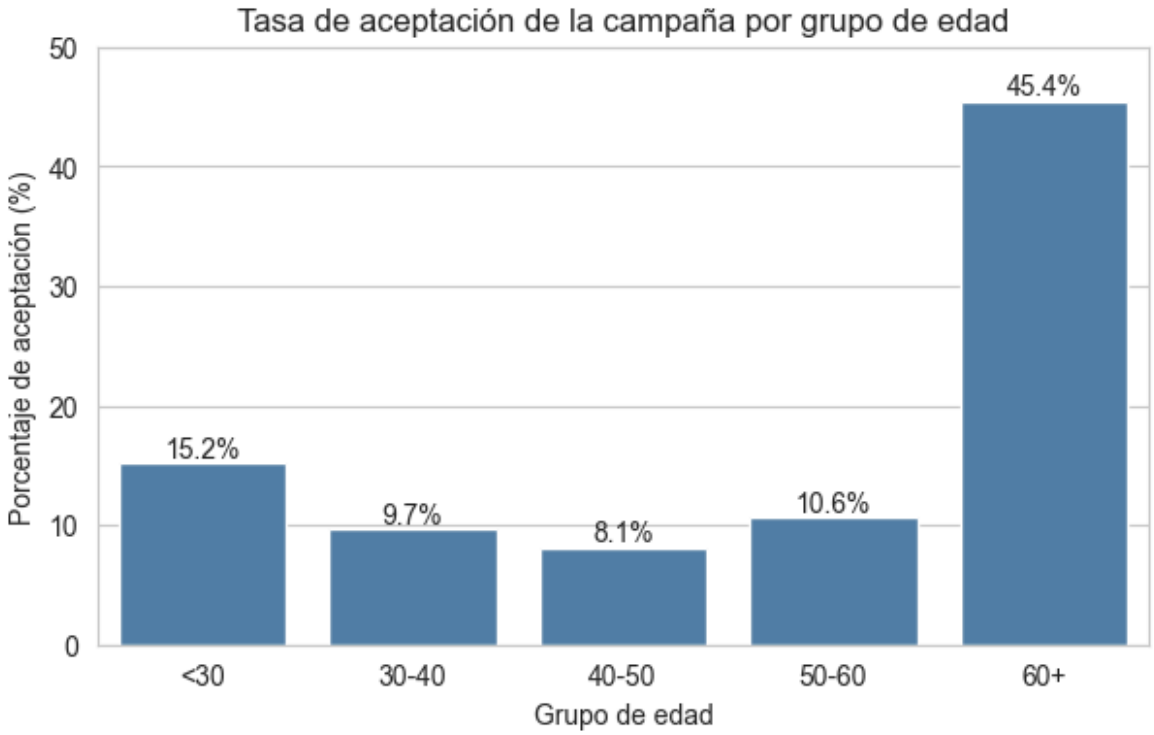
In [ ]:
```
df.groupby("grupo_edad")["y_bin"].agg(["count","mean"])
```

Out[ ]:

grupo_edad	count	mean
<30	7405	0.151789
30-40	16451	0.097076
40-50	10288	0.081260
50-60	6296	0.106099
60+	911	0.454446

In [ ]:

```
# tasa de aceptación por grupo de edad
aceptacion_edad = (
    df.groupby("grupo_edad")["y_bin"]
    .mean()
    .sort_index() * 100
)

plt.figure(figsize=(7,4))

ax = sns.barplot(
    x=aceptacion_edad.index,
    y=aceptacion_edad.values,
    color="steelblue"
)

plt.title("Tasa de aceptación de la campaña por grupo de edad")
plt.xlabel("Grupo de edad")
plt.ylabel("Porcentaje de aceptación (%)")

# agregar porcentaje sobre cada barra
for i, v in enumerate(aceptacion_edad.values):
    ax.text(i, v + 0.5, f"{v:.1f}%", ha="center")

plt.ylim(0,50)

plt.show()
```



Tasa de aceptación según grupo de edad

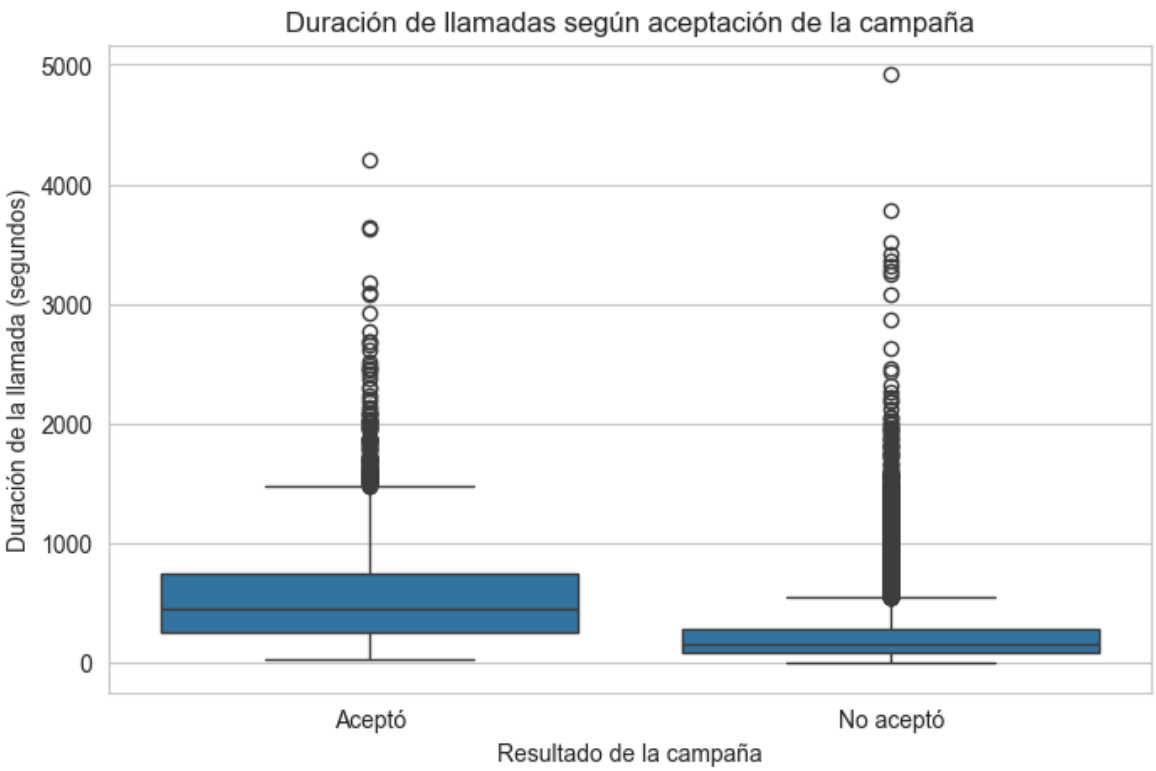
Además de analizar la distribución general de edades, se evaluó la tasa de aceptación de la campaña dentro de cada grupo de edad.

Los resultados muestran que existen diferencias entre segmentos de edad en la probabilidad de aceptar el depósito a plazo. En particular, el grupo de clientes mayores de 60 años presenta una tasa de aceptación considerablemente más alta que el resto de los grupos.

Sin embargo, este resultado debe interpretarse con cautela, ya que el número de clientes en este segmento es significativamente menor en comparación con otros grupos etarios. Por lo tanto, la tasa elevada podría estar influenciada por el tamaño reducido de la muestra.

En general, el análisis sugiere que la edad por sí sola no parece ser un factor determinante en la aceptación de la campaña, aunque ciertos segmentos específicos podrían mostrar mayor receptividad a este tipo de productos financieros.

In [ ]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    x="resultado_campana",
    y="duration",
    data=df
)

plt.title("Duración de llamadas según aceptación de la campaña")
plt.xlabel("Resultado de la campaña")
plt.ylabel("Duración de la llamada (segundos)")

plt.show()
```



Duración de la llamada y aceptación de la campaña

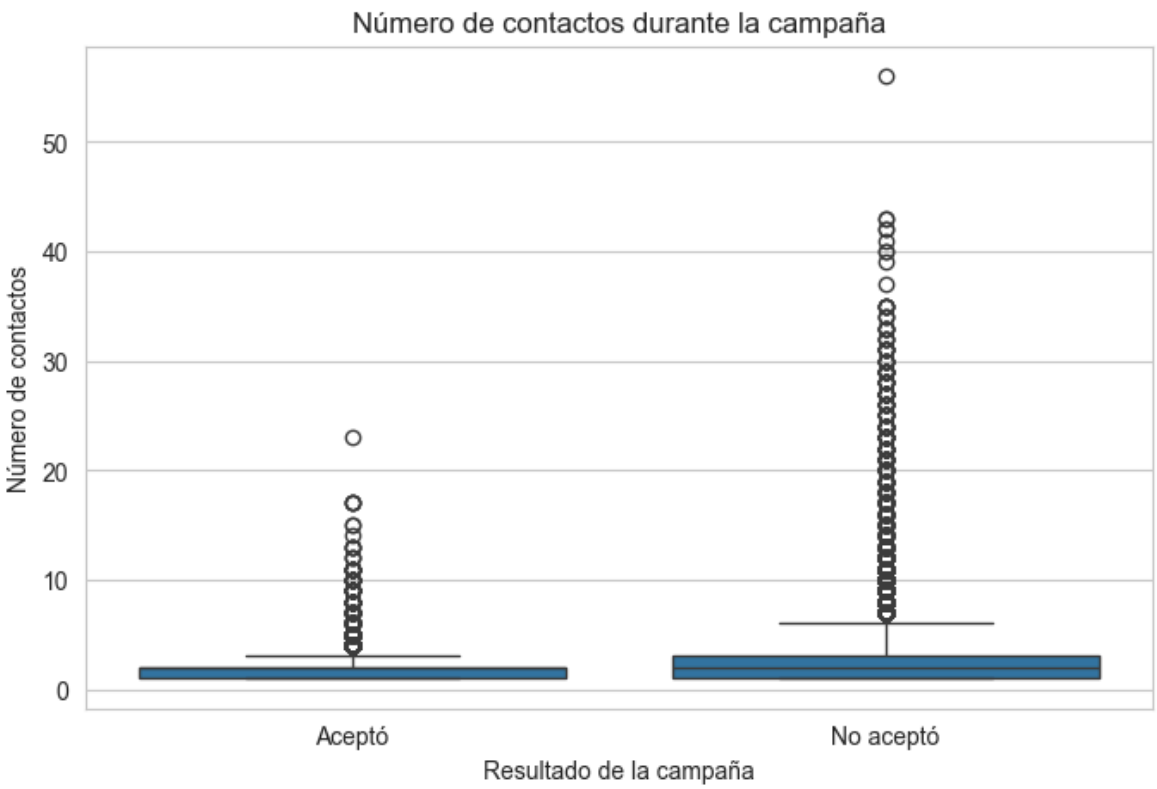
El análisis muestra que las llamadas que terminan en aceptación del depósito a plazo tienden a ser considerablemente más largas que aquellas en las que el cliente rechaza la oferta.

Esto sugiere que cuando el cliente permanece más tiempo en la conversación con el agente existe una mayor probabilidad de conversión.

Sin embargo, es importante considerar que la variable **duration** se registra al finalizar la llamada, por lo que no puede utilizarse como variable predictiva antes de realizar el contacto con el cliente. En otras palabras, la duración de la llamada refleja el resultado de la interacción más que una característica previa del cliente.

Por esta razón, aunque la variable duration muestra una fuerte relación con la aceptación del producto, su utilidad principal es descriptiva y no predictiva.

In [ ]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    x="resultado_campana",
    y="campaign",
    data=df
)

plt.title("Número de contactos durante la campaña")
plt.xlabel("Resultado de la campaña")
plt.ylabel("Número de contactos")

plt.show()
```



Pregunta 2: ¿El número de contactos afecta la probabilidad de conversión?

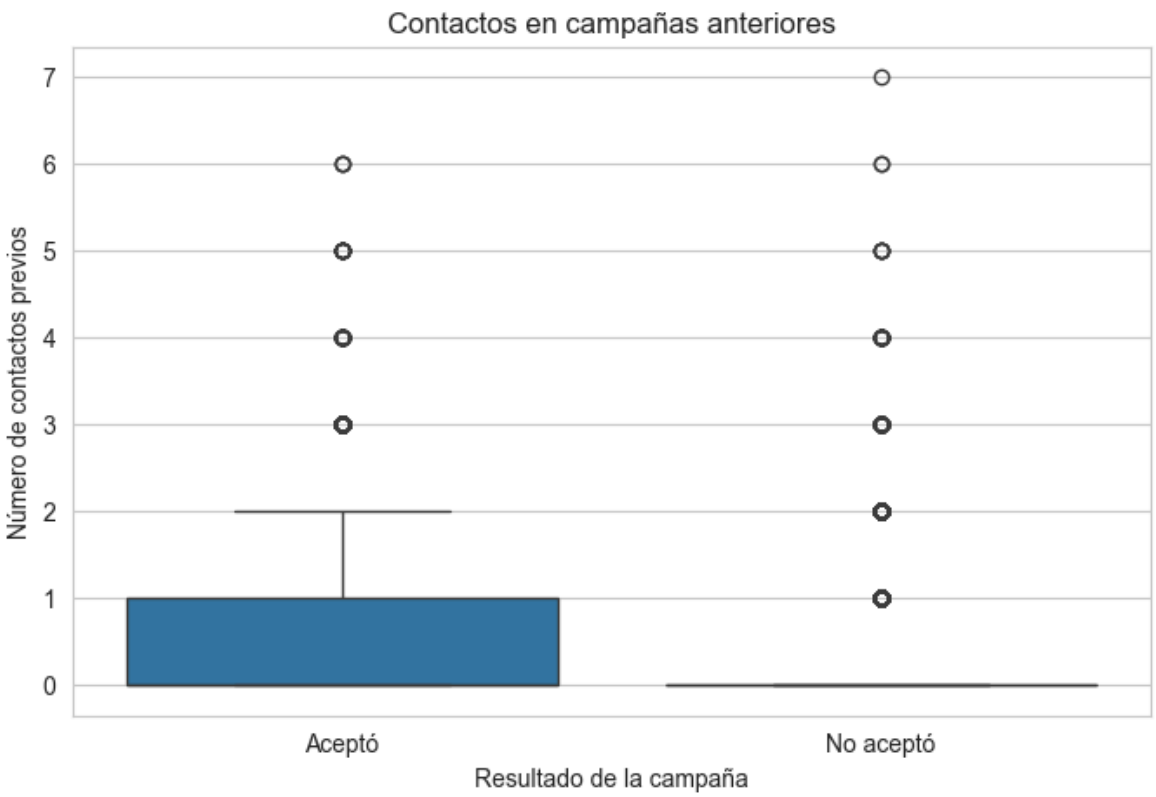
En esta sección se analiza si la cantidad de contactos realizados durante la campaña está asociada con la aceptación del depósito a plazo.

Número de contactos durante la campaña

El análisis del número de contactos realizados durante la campaña muestra que, en general, los clientes que aceptaron el producto tienden a haber recibido menos contactos.

Esto podría indicar que los clientes interesados suelen aceptar la oferta en los primeros intentos, mientras que aquellos que requieren muchos contactos tienden a rechazar el producto.

In [ ]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    x="resultado_campana",
    y="previous",
    data=df
)

plt.title("Contactos en campañas anteriores")
plt.xlabel("Resultado de la campaña")
plt.ylabel("Número de contactos previos")

plt.show()
```



In [ ]:
```
pd.crosstab(df["contactado_previamente"], df["y"], normalize="index") * 1
```

Out[ ]:

y	no	yes
contactado_previamente		
0	91.206452	8.793548
1	73.436116	26.563884

Experiencia previa con campañas del banco

El gráfico muestra el número de contactos realizados en campañas anteriores según el resultado de la campaña actual.

Se observa que algunos clientes que aceptan el producto ya habían sido contactados previamente por el banco, lo que podría indicar que la familiaridad con campañas anteriores influye en la decisión de aceptar la oferta.

In [ ]:
```
tasa_campaign = pd.crosstab(
    df["campaign"],
    df["resultado_campana"],
    normalize="index"
) * 100

tasa_campaign = tasa_campaign[["Aceptó"]]

plt.figure(figsize=(8,5))

sns.lineplot(
    x=tasa_campaign.index,
    y=tasa_campaign["Aceptó"]
```

```
)

plt.title("Tasa de aceptación según número de contactos")
plt.xlabel("Número de contactos realizados")
plt.ylabel("Porcentaje de aceptación")

plt.show()
```

Número de contactos y probabilidad de aceptación

El análisis de la tasa de aceptación según el número de contactos realizados durante la campaña muestra que la probabilidad de conversión tiende a disminuir a medida que aumenta el número de intentos.

Esto sugiere que los clientes que están interesados en el producto suelen aceptarlo en los primeros contactos. En cambio, cuando se requieren muchos intentos de contacto, la probabilidad de aceptación tiende a ser menor.

Desde una perspectiva de negocio, este resultado podría indicar que insistir repetidamente con clientes poco interesados no necesariamente mejora la efectividad de la campaña.

In [ ]:
```
aceptaron = df[df["y"] == "yes"]

print("Promedio de contactos en clientes que aceptaron:", round(aceptaron
print("Mediana de contactos en clientes que aceptaron:", aceptaron["campa
```

```
Promedio de contactos en clientes que aceptaron: 2.05
Mediana de contactos en clientes que aceptaron: 2.0
```

In [ ]:
```
limite = 3

df["llamadas_extra"] = (df["campaign"] - limite).clip(lower=0)
```

```
total_llamadas_extra = df["llamadas_extra"].sum()

print("Total de llamadas adicionales por encima de 3 intentos:", total_ll
```

```
Total de llamadas adicionales por encima de 3 intentos: 28249
```

In [ ]:
```
costo_por_llamada = 1.5

costo_estimado = total_llamadas_extra * costo_por_llamada

print("Costo operativo estimado de llamadas adicionales: $", round(costo_
```

```
Costo operativo estimado de llamadas adicionales: $ 42373.5
```

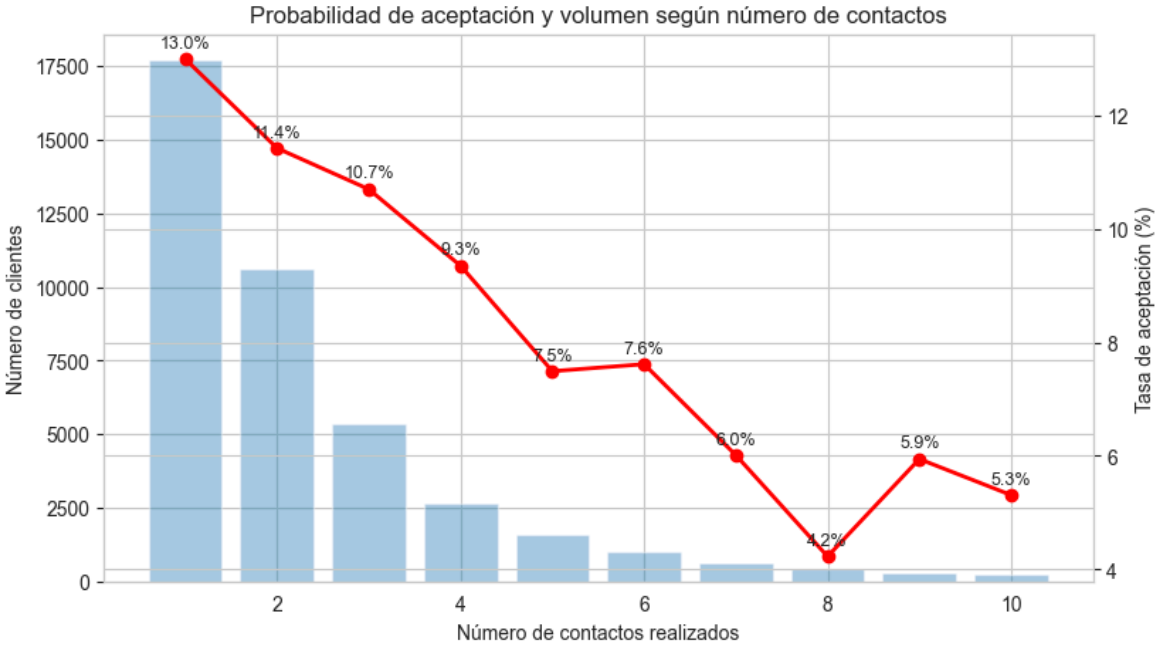
In [ ]:
```
# resumen de contactos
resumen_contactos = df.groupby("campaign")["y_bin"].agg(
    total_clientes="count",
    tasa_aceptacion="mean"
)

# convertir tasa a porcentaje
resumen_contactos["tasa_aceptacion"] = resumen_contactos["tasa_aceptacion

# ver solo primeros contactos
resumen_contactos.loc[1:10]
```

Out[ ]:

campaign	total_clientes	tasa_aceptacion
1	17703	12.986499
2	10606	11.418065
3	5367	10.694988
4	2664	9.346847
5	1602	7.490637
6	985	7.614213
7	632	6.012658
8	402	4.228856
9	286	5.944056
10	226	5.309735

In [ ]:
```
import matplotlib.pyplot as plt

# limitar a primeros contactos
data_plot = resumen_contactos.loc[1:10]

fig, ax1 = plt.subplots(figsize=(9,5))

# barras = volumen de clientes
ax1.bar(
    data_plot.index,
    data_plot["total_clientes"],
    alpha=0.4,
```

```
    label="Número de clientes"
)

ax1.set_xlabel("Número de contactos realizados")
ax1.set_ylabel("Número de clientes")

# segunda escala para tasa
ax2 = ax1.twinx()

ax2.plot(
    data_plot.index,
    data_plot["tasa_aceptacion"],
    marker="o",
    linewidth=2,
    color="red",
    label="Tasa de aceptación (%)"
)

ax2.set_ylabel("Tasa de aceptación (%)")

# ✨ agregar etiquetas de porcentaje
for x, y in zip(data_plot.index, data_plot["tasa_aceptacion"]):
    ax2.text(
        x,
        y + 0.2,
        f"{y:.1f}%",
        ha="center",
        fontsize=9
    )

plt.title("Probabilidad de aceptación y volumen según número de contactos

plt.show()
```



Eficiencia operativa del número de contactos

Además de analizar la tasa de aceptación según el número de contactos, se evaluó en qué intento promedio se logra la conversión y cuántos intentos adicionales se

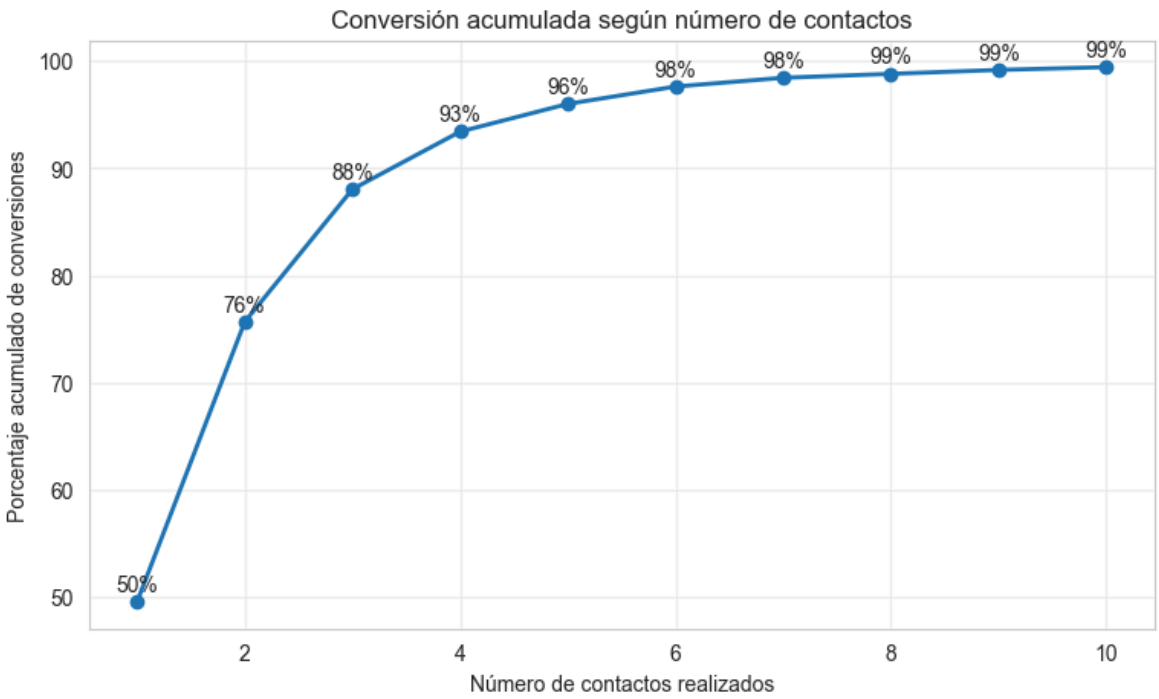
realizan por cliente.

Los resultados muestran que la mayoría de las conversiones ocurre en los primeros contactos. A partir de ese punto, los intentos adicionales aportan un valor marginal bajo.

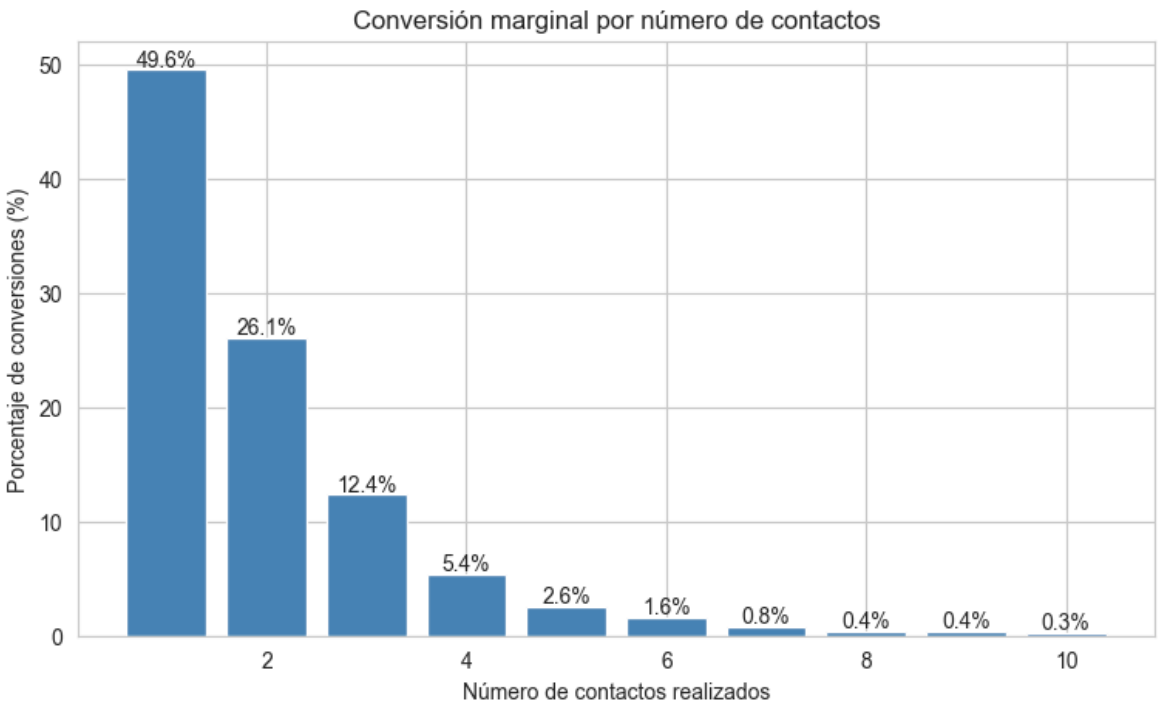
Bajo un supuesto de costo por llamada, los contactos realizados por encima de un límite operativo podrían representar un costo adicional significativo para la campaña.

Este análisis sugiere que establecer un límite máximo de intentos de contacto podría mejorar la eficiencia operativa de las campañas telefónicas.

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

# filtrar clientes que aceptaron
aceptaron = df[df["y"] == "yes"]

# contar conversiones por número de contactos
conv_contactos = aceptaron.groupby("campaign").size().sort_index()

# acumulado
conv_acumulado = conv_contactos.cumsum()

# porcentaje acumulado
conv_pct = conv_acumulado / conv_contactos.sum() * 100

# limitar a 10 contactos
conv_pct = conv_pct.loc[:10]

plt.figure(figsize=(9,5))

plt.plot(
    conv_pct.index,
    conv_pct.values,
    marker="o",
    linewidth=2
)

plt.title("Conversión acumulada según número de contactos")
plt.xlabel("Número de contactos realizados")
plt.ylabel("Porcentaje acumulado de conversiones")

for x, y in zip(conv_pct.index, conv_pct.values):
    plt.text(x, y+1, f"{y:.0f}%", ha="center")

plt.grid(alpha=0.3)

plt.show()
```



In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

# filtrar clientes que aceptaron
aceptaron = df[df["y"] == "yes"]

# contar conversiones por contacto
conv_contactos = aceptaron.groupby("campaign").size().sort_index()

# porcentaje marginal
conv_marginal = conv_contactos / conv_contactos.sum() * 100

# limitar a 10 contactos
conv_marginal = conv_marginal.loc[:10]

plt.figure(figsize=(9,5))

plt.bar(
    conv_marginal.index,
    conv_marginal.values,
    color="steelblue"
)

plt.title("Conversión marginal por número de contactos")
plt.xlabel("Número de contactos realizados")
plt.ylabel("Porcentaje de conversiones (%)")

for x, y in zip(conv_marginal.index, conv_marginal.values):
    plt.text(x, y+0.3, f"{y:.1f}%", ha="center")

plt.show()
```

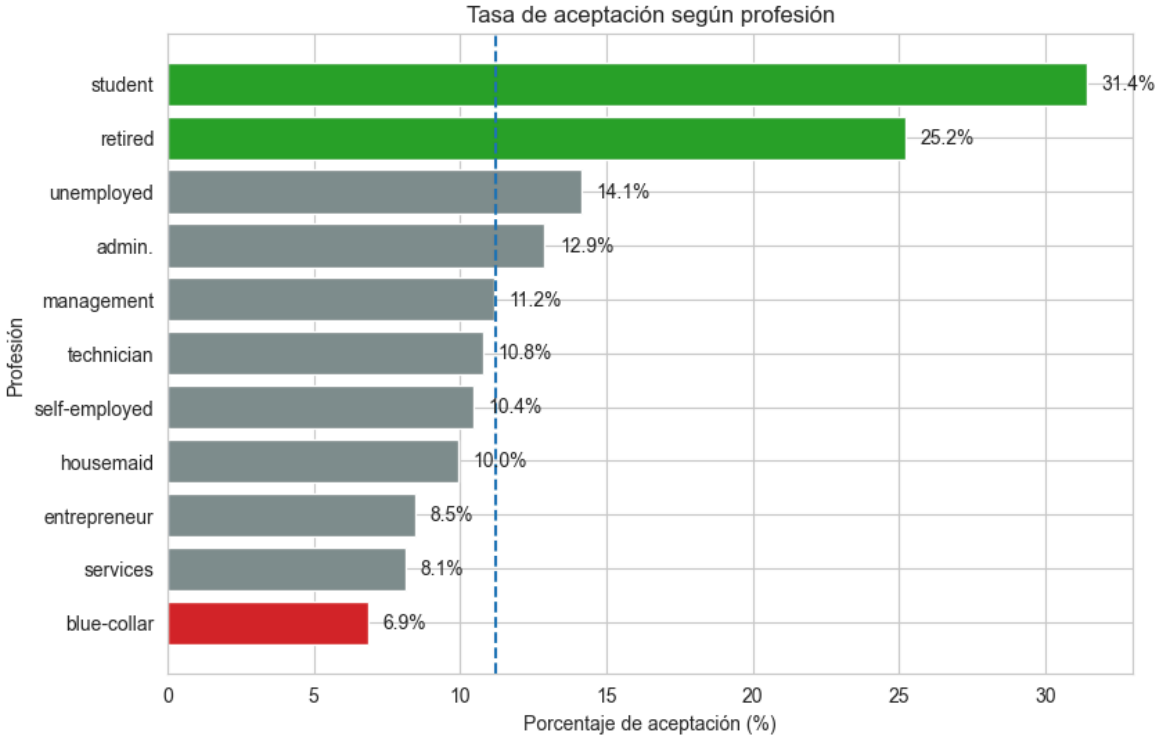


Pregunta 3: ¿La profesión del cliente está asociada con mayores tasas de aceptación?

Se analiza la tasa de aceptación del depósito a plazo según la profesión del cliente para identificar posibles oportunidades de segmentación.

In [ ]:
```
# tasa de aceptación por profesión
tasa_job = pd.crosstab(
    df["job"],
    df["resultado_campana"],
    normalize="index"
) * 100

tasa_job = tasa_job[["Aceptó"]].sort_values(by="Aceptó", ascending=True)

# promedio general
promedio = df["resultado_campana"].value_counts(normalize=True)["Aceptó"]

# colores según desempeño
colors = []

for v in tasa_job["Aceptó"]:
    if v >= 20:
        colors.append("#2ca02c")   # verde = alto desempeño
    elif v <= 8:
        colors.append("#d62728")   # rojo = bajo desempeño
    else:
        colors.append("#7f8c8d")   # gris = promedio

plt.figure(figsize=(9,6))

ax = plt.barh(
    tasa_job.index,
    tasa_job["Aceptó"],
    color=colors
)
```

```
plt.title("Tasa de aceptación según profesión")
plt.xlabel("Porcentaje de aceptación (%)")
plt.ylabel("Profesión")

# línea de promedio
plt.axvline(promedio, linestyle="--")

# etiquetas de porcentaje
for i, v in enumerate(tasa_job["Aceptó"]):
    plt.text(v + 0.5, i, f"{v:.1f}%", va="center")

plt.show()
```



Profesión y aceptación de la campaña

Se analizó la relación entre la profesión del cliente y la aceptación del depósito a plazo con el objetivo de identificar posibles oportunidades de segmentación.

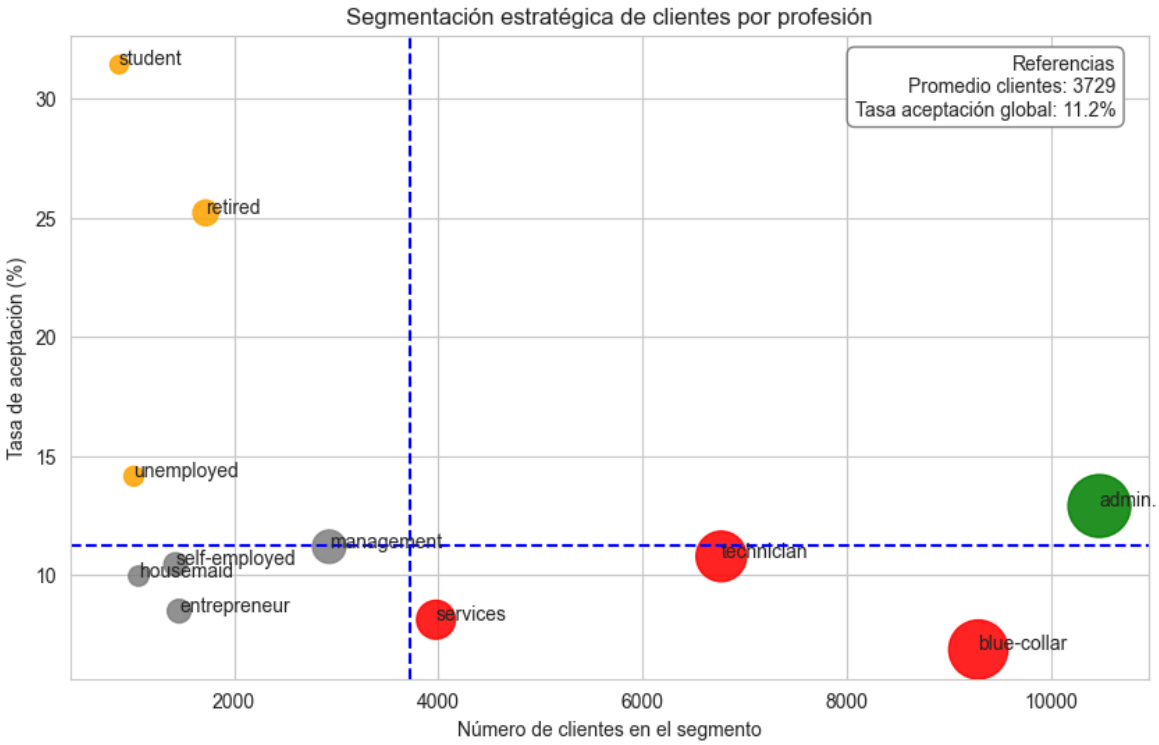
Los resultados muestran diferencias relevantes entre profesiones. Por ejemplo, los **estudiantes presentan la mayor tasa de aceptación (31.4%)**, seguidos por **clientes retirados (25.2%)** y **personas desempleadas (14.1%)**. En contraste, profesiones como **blue-collar (6.9%)** y **services (8.1%)** presentan menores tasas de conversión.

Estos resultados sugieren que la ocupación del cliente puede estar asociada con distintos niveles de interés en productos de ahorro o inversión.

Desde una perspectiva de negocio, este tipo de información puede ayudar al banco a **segmentar mejor sus campañas comerciales**, priorizando perfiles con mayor probabilidad de conversión y optimizando el uso de los recursos de contacto.

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

# Agrupar por profesión
segmentos = df.groupby("job").agg(
    clientes=("job","count"),
    tasa_aceptacion=("y_bin","mean")
).reset_index()

# Convertir a porcentaje
segmentos["tasa_aceptacion"] = segmentos["tasa_aceptacion"]*100

# Promedios correctos
promedio_clientes = segmentos["clientes"].mean()
promedio_aceptacion = df["y_bin"].mean()*100   # tasa global real

# Tamaño de burbujas
sizes = segmentos["clientes"]/10

# Colores por cuadrante
colors = []

for _, row in segmentos.iterrows():

    if row["tasa_aceptacion"] >= promedio_aceptacion and row["clientes"]
        colors.append("orange")   # nichos atractivos

    elif row["tasa_aceptacion"] >= promedio_aceptacion and row["clientes"
        colors.append("green")    # estratégicos

    elif row["tasa_aceptacion"] < promedio_aceptacion and row["clientes"]
        colors.append("red")      # ineficientes

    else:
        colors.append("gray")     # baja prioridad


plt.figure(figsize=(10,6))

plt.scatter(
    segmentos["clientes"],
    segmentos["tasa_aceptacion"],
    s=sizes,
    c=colors,
    alpha=0.85
)

# etiquetas de cada profesión
for i,row in segmentos.iterrows():
    plt.text(row["clientes"],row["tasa_aceptacion"],row["job"])

# líneas de referencia
plt.axvline(promedio_clientes, linestyle="--", color="blue")
plt.axhline(promedio_aceptacion, linestyle="--", color="blue")

# cuadro informativo con promedios
texto = (
    f"Referencias\n"
    f"Promedio clientes: {round(promedio_clientes)}\n"
```

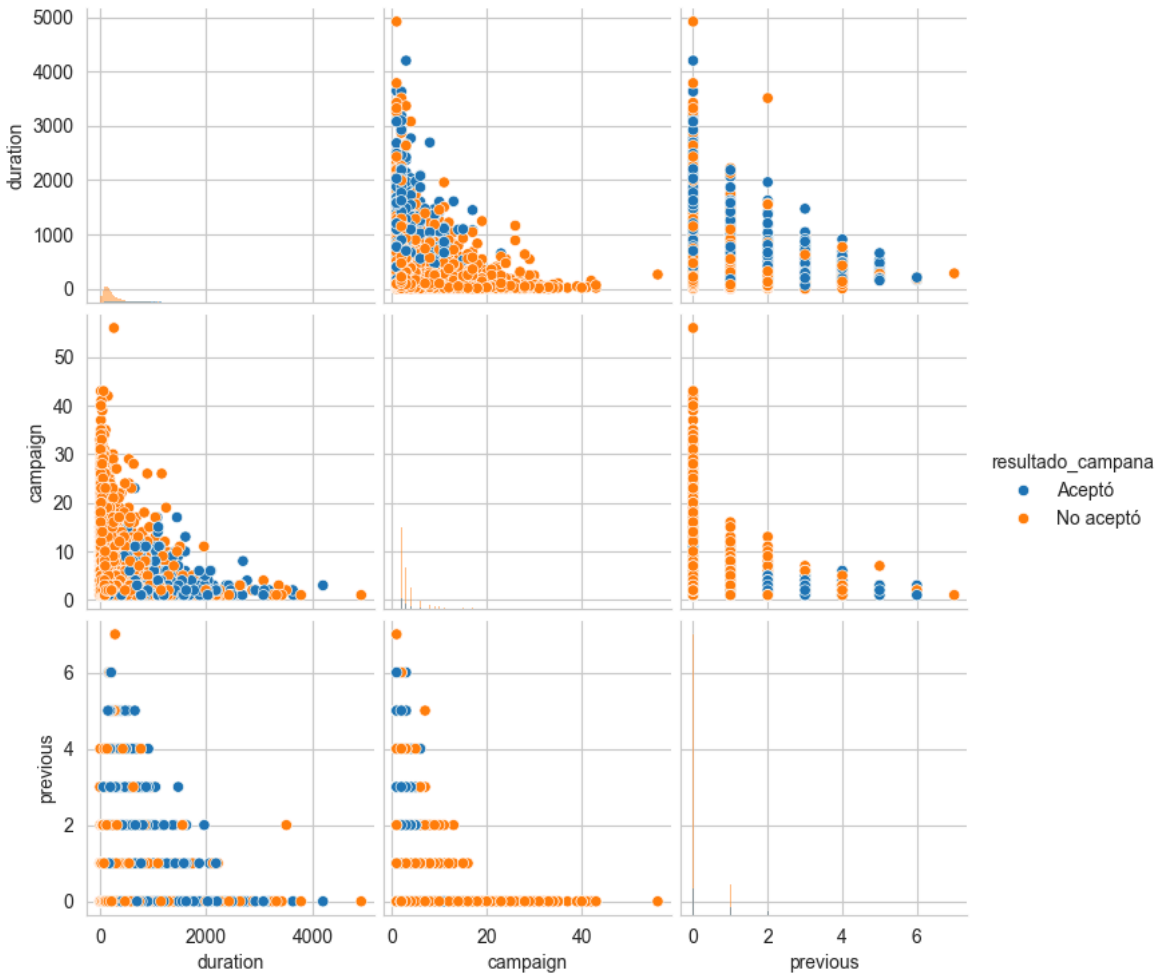
```
    f"Tasa aceptación global: {round(promedio_aceptacion,1)}%"
)

plt.gca().text(
    0.97,0.97, texto,
    transform=plt.gca().transAxes,
    verticalalignment="top",
    horizontalalignment="right",
    fontsize=10,
    bbox=dict(boxstyle="round,pad=0.4", facecolor="white", edgecolor="gra
)

plt.xlabel("Número de clientes en el segmento")
plt.ylabel("Tasa de aceptación (%)")
plt.title("Segmentación estratégica de clientes por profesión")

plt.show()
```



In [ ]:
```
df["resultado_campana"] = df["y_bin"].map({1:"Aceptó", 0:"No aceptó"})
```

In [ ]:
```
sns.pairplot(
    df[["duration","campaign","previous","resultado_campana"]],
    hue="resultado_campana",
    diag_kind="hist"
)
```

Out[ ]:
```
<seaborn.axisgrid.PairGrid at 0x11bcc47d0>
```



Exploración multivariable

El pairplot permite observar simultáneamente la relación entre varias variables clave del dataset.

Se observa que la duración de la llamada parece diferenciar claramente entre clientes que aceptaron y aquellos que no aceptaron la campaña. En cambio, variables como el número de contactos durante la campaña o en campañas anteriores muestran patrones menos claros.

Conclusión del análisis descriptivo

El análisis descriptivo permite identificar algunos patrones relevantes en el comportamiento de los clientes frente a la campaña de marketing.

En particular, la duración de la llamada aparece como el factor más asociado con la aceptación del depósito a plazo. Asimismo, se observa que los clientes que aceptan el producto suelen hacerlo en los primeros contactos, mientras que un mayor número de intentos tiende a estar asociado con una menor probabilidad de conversión.

Estos resultados ayudan a comprender mejor la dinámica de la campaña y permiten identificar variables potencialmente relevantes para futuros modelos predictivos.

Insights Adicionales del análisis

Insights estratégicos del análisis

Identificación de perfiles con mayor probabilidad de aceptación

Con el objetivo de identificar segmentos de clientes con mayor probabilidad de aceptar el depósito a plazo, se analizó la combinación entre variables demográficas como la profesión del cliente y su grupo de edad.

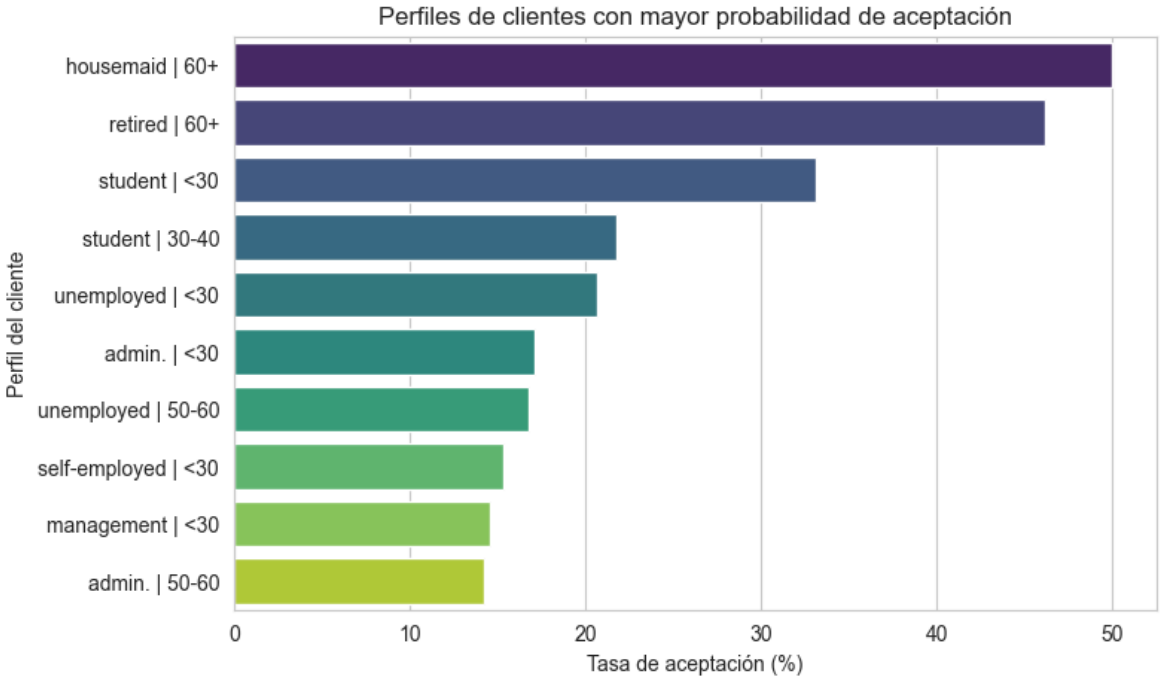
Para evitar sesgos asociados a segmentos con muy pocos clientes, se consideraron únicamente aquellos perfiles que cuentan con **al menos 50 observaciones**. Este criterio permite asegurar que las tasas de aceptación calculadas sean más representativas y no estén influenciadas por tamaños de muestra muy pequeños.

El gráfico presenta los perfiles que muestran mayores tasas de aceptación dentro de estos segmentos representativos. Este tipo de análisis permite identificar grupos de clientes potencialmente más receptivos a la oferta del banco y puede ser útil para orientar futuras estrategias de segmentación comercial y optimizar la eficiencia de las campañas de marketing.

In [ ]:

In [ ]:
```
# Crear tabla de aceptación por perfil
perfil = pd.crosstab(
    [df["job"], df["grupo_edad"]],
    df["y"]
)

# Calcular total de clientes por perfil
perfil["total"] = perfil.sum(axis=1)

# Calcular tasa de aceptación
perfil["tasa_aceptacion"] = (perfil["yes"] / perfil["total"]) * 100

# Filtrar perfiles con número mínimo de clientes
perfil_filtrado = perfil[perfil["total"] >= 50]

# Ordenar por tasa de aceptación
perfil_filtrado = perfil_filtrado.sort_values(
    by="tasa_aceptacion",
    ascending=False
)

# Seleccionar los 10 perfiles con mayor tasa
top_perfiles = perfil_filtrado.head(10).reset_index()

# Crear etiqueta del perfil
top_perfiles["perfil"] = (
    top_perfiles["job"].astype(str) +
```

```
    " | " +
    top_perfiles["grupo_edad"].astype(str)
)

# Gráfico de perfiles con mayor probabilidad de aceptación
plt.figure(figsize=(8,5))

sns.barplot(
    x="tasa_aceptacion",
    y="perfil",
    data=top_perfiles,
    hue="perfil",
    legend=False,
    palette="viridis"
)

plt.title("Perfiles de clientes con mayor probabilidad de aceptación")
plt.xlabel("Tasa de aceptación (%)")
plt.ylabel("Perfil del cliente")

plt.show()
```



Parte 5: Conclusiones y recomendaciones

Conclusiones

El análisis de las campañas telefónicas permitió identificar algunos patrones relevantes en la probabilidad de aceptación del depósito a plazo.

- La tasa de conversión de la campaña es aproximadamente **11%**, lo que indica que la mayoría de los contactos realizados no generan una venta.
- Se observa que **la mayoría de las conversiones ocurre en los primeros contactos**, con un promedio cercano a **2 intentos por cliente que finalmente**

acepta la oferta.

- El análisis muestra que **aumentar el número de intentos de contacto no mejora significativamente la probabilidad de conversión**, lo que sugiere retornos decrecientes en la insistencia comercial.
- También se observaron diferencias en la probabilidad de aceptación entre distintos segmentos de clientes, particularmente según variables como **la edad y la profesión**.
- Además, **las llamadas que terminan en conversión tienden a tener mayor duración**, lo que sugiere que los clientes que muestran mayor interés durante la conversación tienen mayor probabilidad de aceptar el producto.

Recomendaciones estratégicas

- Implementar **criterios de segmentación de clientes antes de realizar campañas**, priorizando perfiles con mayor probabilidad de conversión.
- Establecer un **límite operativo de aproximadamente tres intentos de contacto por cliente**, ya que la mayor parte de las conversiones ocurre antes de ese punto.
- Reducir contactos adicionales que generan bajo retorno, ya que el análisis muestra que **los intentos por encima de ese umbral representan miles de llamadas adicionales con impacto operativo en costos de campaña**.
- Utilizar estos resultados como base para desarrollar **modelos predictivos de propensión a compra**, que permitan identificar clientes con mayor probabilidad de aceptar la oferta antes de iniciar nuevas campañas.

En conjunto, estas acciones permitirían **optimizar los recursos del equipo comercial y mejorar la eficiencia de las campañas del banco**.

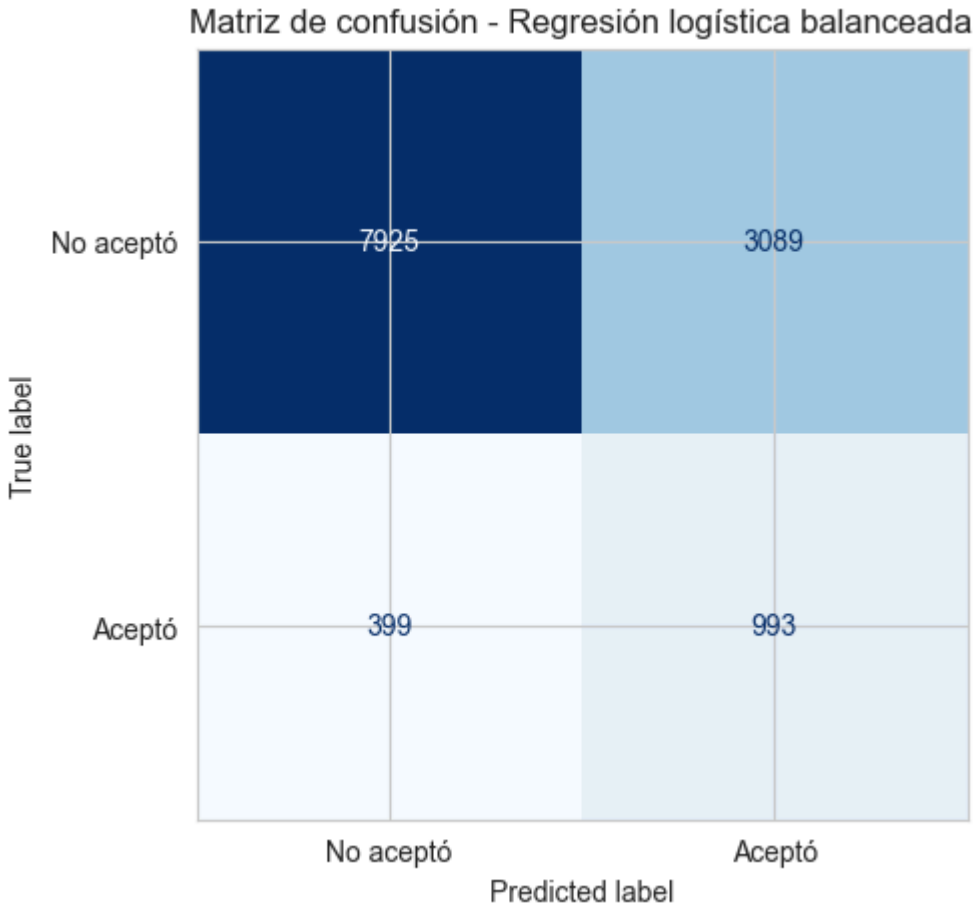
Extensión del análisis: modelo de propensión a aceptación

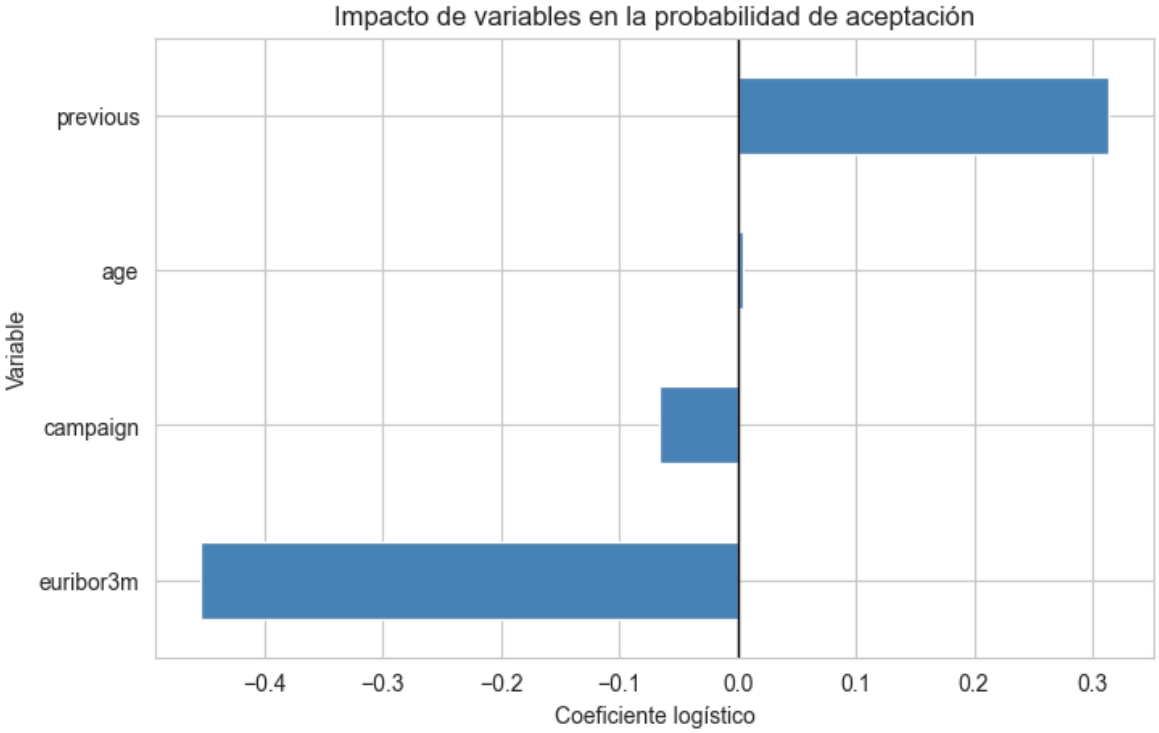
In [ ]:
```
# ========================================
# Extensión del análisis: modelo predictivo
# Regresión logística con clases balanceadas
# ========================================

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, Conf

# ----------------------------
# 1. Variables predictoras
```

```
# ------------------------------
# Puedes ajustar estas variables si luego quieres probar más features
X = df[["age", "campaign", "previous", "euribor3m"]]
y = df["y_bin"]

# ------------------------------
# 2. División train / test
# ------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# ------------------------------
# 3. Modelo logístico balanceado
# ------------------------------
modelo = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    random_state=42
)

modelo.fit(X_train, y_train)

# ------------------------------
# 4. Predicciones
# ------------------------------
y_pred = modelo.predict(X_test)

# Probabilidad estimada de aceptar la campaña
y_prob = modelo.predict_proba(X_test)[:, 1]

# ------------------------------
# 5. Resultados del modelo
# ------------------------------
print("=== Classification Report ===")
print(classification_report(y_test, y_pred))

# ------------------------------
# 6. Matriz de confusión
# ------------------------------
cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(6, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No ac
disp.plot(ax=ax, cmap="Blues", colorbar=False)
plt.title("Matriz de confusión - Regresión logística balanceada")
plt.show()

# ------------------------------
# 7. Coeficientes del modelo
# ------------------------------
coef = pd.Series(modelo.coef_[0], index=X.columns).sort_values()

plt.figure(figsize=(8, 5))
coef.plot(kind="barh", color="steelblue")
plt.axvline(0, color="black", linewidth=1)
plt.title("Impacto de variables en la probabilidad de aceptación")
```

```
plt.xlabel("Coeficiente logístico")
plt.ylabel("Variable")
plt.show()

# ------------------------------
# 8. Tabla resumen de coeficientes
# ------------------------------
coef_df = pd.DataFrame({
    "Variable": coef.index,
    "Coeficiente": coef.values
}).sort_values(by="Coeficiente", ascending=False)

print("=== Coeficientes del modelo ===")
print(coef_df)

# ------------------------------
# 9. Probabilidades estimadas (opcional)
# ------------------------------
resultado_prob = X_test.copy()
resultado_prob["real"] = y_test.values
resultado_prob["prediccion"] = y_pred
resultado_prob["prob_aceptacion"] = y_prob

print("\n=== Top 10 clientes con mayor probabilidad estimada de aceptació
print(resultado_prob.sort_values(by="prob_aceptacion", ascending=False).h
```

```
=== Classification Report ===
              precision    recall  f1-score   support

           0       0.95      0.72      0.82     11014
           1       0.24      0.71      0.36      1392

    accuracy                           0.72     12406
   macro avg       0.60      0.72      0.59     12406
weighted avg       0.87      0.72      0.77     12406
```


Matriz de confusión - Regresión logística balanceada
No aceptó
Aceptó
True label
7925
3089
399
993
No aceptó
Aceptó
Predicted label


Impacto de variables en la probabilidad de aceptación
previous
age
campaign
euribor3m
Variable
−0.4
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3
Coeficiente logístico

```
=== Coeficientes del modelo ===
    Variable  Coeficiente
3   previous     0.313239
2        age     0.004953
1   campaign    -0.066585
0  euribor3m    -0.454085

=== Top 10 clientes con mayor probabilidad estimada de aceptación ===
       age  campaign  previous  euribor3m  real  prediccion  prob_aceptaci
on
4002    77         1         6      0.905     1           1         0.9524
46
33746   62         1         6      1.031     1           1         0.9461
23
20652   48         3         6      0.904     1           1         0.9382
44
11404   52         1         5      0.677     1           1         0.9348
47
30250   77         1         4      0.682     1           1         0.9221
52
6568    58         1         4      0.683     1           1         0.9150
87
14128   80         3         4      0.649     1           1         0.9144
04
9897    41         1         4      0.683     1           1         0.9083
12
8723    52         1         4      0.883     1           1         0.9052
41
29707   36         1         4      0.720     1           1         0.9047
91
```

Interpretación del modelo predictivo

El modelo de regresión logística muestra una alta precisión al identificar clientes que no aceptan la campaña, lo cual refleja el fuerte desbalance presente en el dataset, donde la mayoría de los clientes no acepta la oferta.

Sin embargo, el modelo presenta una capacidad limitada para identificar correctamente a los clientes que sí aceptan la campaña. Esto sugiere que, para construir un modelo predictivo más efectivo, sería necesario incorporar más variables explicativas o aplicar técnicas adicionales para manejar el desbalance de clases.

Este ejercicio demuestra cómo los insights obtenidos en el análisis exploratorio pueden servir como punto de partida para el desarrollo de modelos de propensión a compra en un entorno real de analítica de marketing.

En conjunto, este análisis permite comprender mejor los factores asociados con la aceptación de las campañas telefónicas y proporciona una base analítica para mejorar la segmentación de clientes y optimizar la eficiencia de futuras estrategias comerciales del banco.

In [ ]:
```
!jupyter nbconvert --to html campana_marketing.ipynb
```

```
[NbConvertApp] Converting notebook campana_marketing.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 23 image(s).
[NbConvertApp] Writing 1602511 bytes to campana_marketing.html
```